



UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

There slides are mainly extracted from:

Frank Tsui, Orlando Karam, Barbara Bernal, Software engineering (Third Edition), Jones & Bartlett Learning
Neal Ford, Mark Richards: Fundamentals of Software Architectures, O'Reilly 2021

Raju Gandhi, Mark Richards, Neal Ford: Head First Software Architecture: A Learner's Guide to Architectural Thinking 1st Edition, 2024

Software architecture and design 2

Francesco Guerra
francesco.guerra@unimore.it

Software Architecture



Software architecture and software design

- ▶ Software Engineer researchers have long argued about a definition of architecture.
 1. It's something like the fundamental organization of a system, or the way the highest level components are wired together.
 - ▶ There is no objective way to define what is fundamental, or high level
 - ▶ The architecture is the shared understanding that the expert developers have of the system design.
 2. It's “the design decisions that need to be made early in a project”.
- ▶ Conclusion:
“Architecture is about the important stuff. Whatever that is”

<https://martinfowler.com/architecture/>

- ▶ Typically for understanding software architecture is used the building metaphor

The dimensions of software architecture

▶ Architectural characteristics

- ▶ This dimension describes what aspects the architecture needs to support: e.g., scalability, testability, availability, ...

▶ Architectural decisions

- ▶ This dimension includes important decisions that have long-term or significant implications for the system: e.g., , the kind of database, the number of services, how services communicate with each other ...

▶ Logical components

- ▶ This dimension describes the building blocks of the system's functionality and how they interact with each other. E.g., an ecommerce system might have components for inventory management, payment processing, ...

▶ Architectural style

- ▶ This dimension defines the physical shape and structure of a software as a building plan defines the overall shape and structure of a home.

Architectural characteristics

- ▶ They form the foundation of the architecture in a software system.
- ▶ **Architectural characteristics** are also known as nonfunctional requirements, system quality attributes, and simply “the -ilities” because most end with the suffix *-ility*.

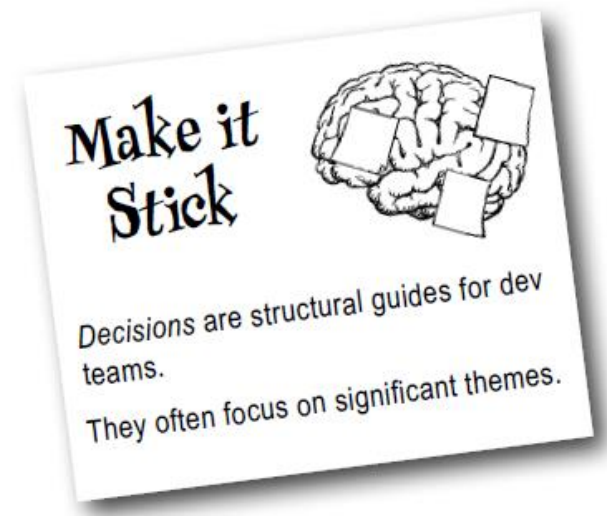


Architectural decisions

- ▶ ***Architectural decisions*** are choices you make about structural aspects of the system that have long-term or significant implications.
 - ▶ As constraints, they guide the development team in planning and building the system.
- ▶ You might decide that your system's user interface
 - ▶ should not communicate directly with the database, but is based on services to retrieve and update data.
- ▶ This architectural decision
 - ▶ places a constraint on the development of the user interface,
 - ▶ guides the development team about how other components should access and update data in the database.

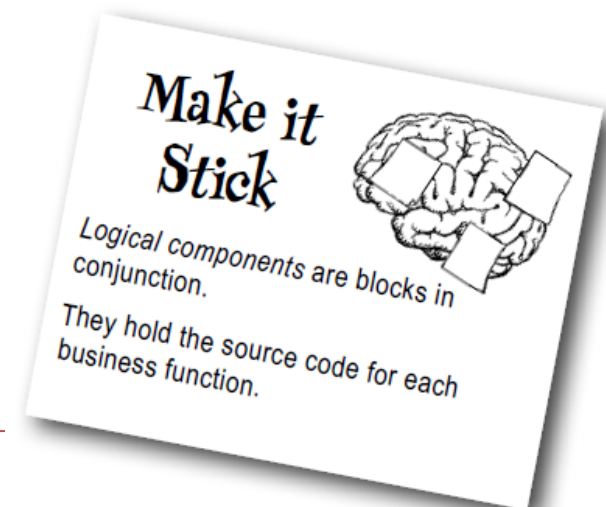
Architectural decisions

- ▶ It's not uncommon to have several dozen or more documented architectural decisions within any system.
- ▶ Generally, the larger and more complicated the system, the more architectural decisions it will have.



Logical components

- ▶ **Logical components** are the building blocks of a system.
- ▶ A logical component performs some sort of function, such as processing the payment for an order.
 - ▶ Logical components in a system are usually represented through a directory or namespace.
 - ▶ For example, the directory `app/order/payment` with the corresponding namespace `app.order`.
 - ▶ `payment` identifies a logical component named Payment Processing.
 - ▶ The source code that allows users to pay for an order is stored in this directory and uses this namespace



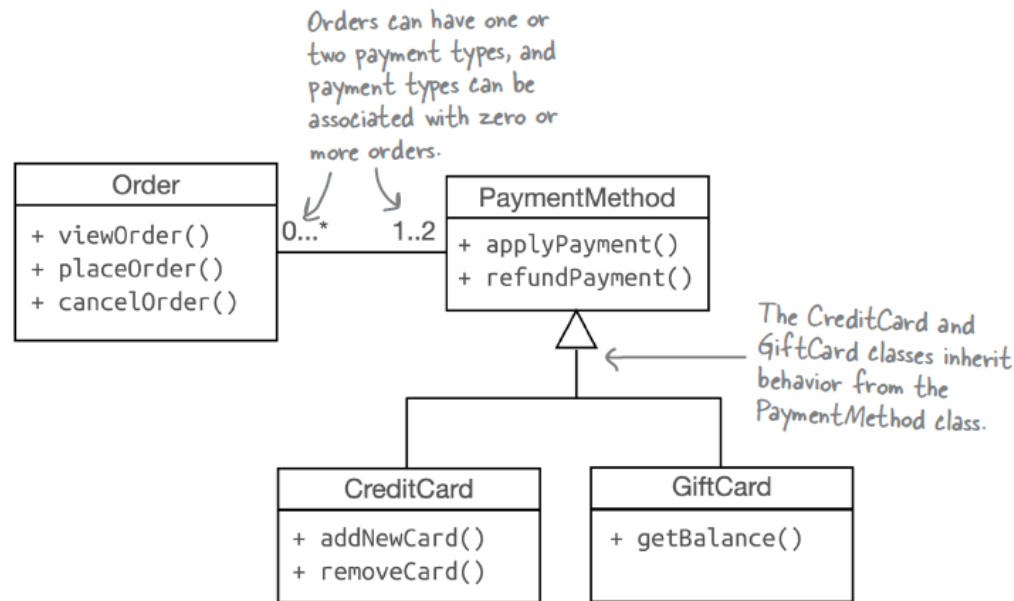
Architectural styles

- ▶ Architectural styles define the overall shape and structure of a software system, each with its own unique set of characteristics.
 - ▶ The *microservices* architectural style scales very well and provides a high level of *agility*.
 - ▶ The *layered* architectural style is less complex and less costly.
 - ▶ The *event-driven* architectural style provides high levels of scalability and is very fast and responsive.



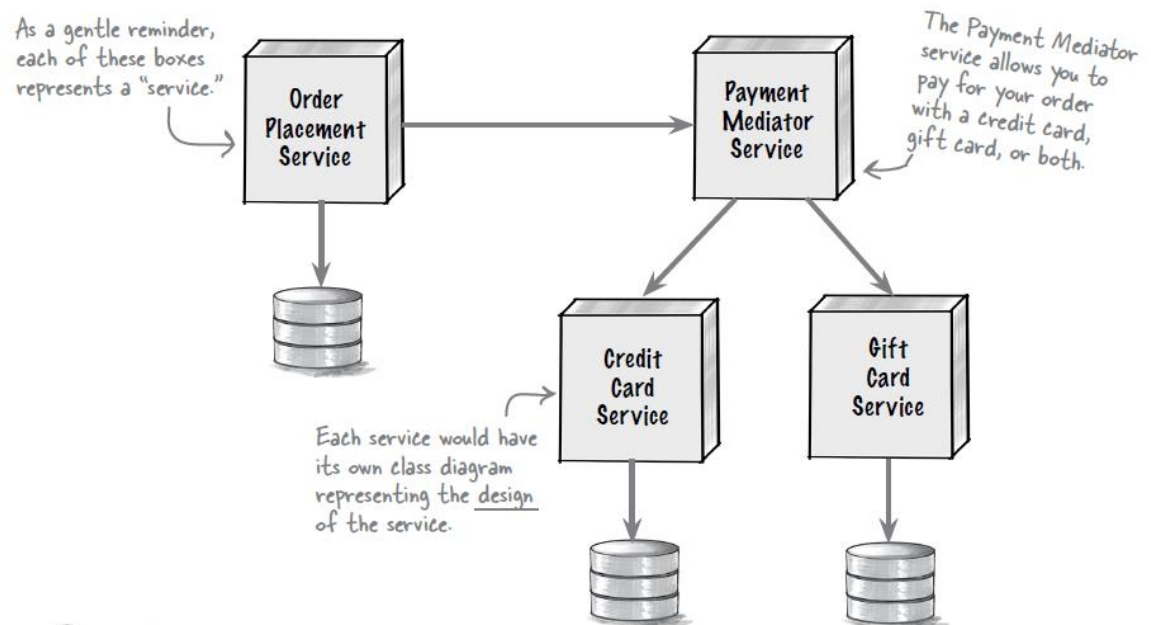
Design vs Architecture

- ▶ From a *design* perspective, you might build a UML class diagram to show how the classes interact to implement the functionality.
- ▶ The design says nothing about the *physical structure* of the source code, i.e., how these class files would be organized and deployed



Design vs Architecture

- ▶ Architecture is about the *structure* of the system—things like services, databases, and how services communicate with each other and the user interface
- ▶ From an *architectural* perspective, you might decide to create separate services for each payment type within the order payment process and have an orchestrator service to manage the payment processing part of the system, like in the diagram below.



Design vs Architecture

- ▶ Some decisions are architectural (deciding which architectural style to use)
- ▶ Others are design-related (such as changing the position of a field on a screen or changing the type of a field within a class).
- ▶ Most decisions will fall within a *spectrum* of architecture and design.
- ▶ Strategic or tactical decision?
 - ▶ **Strategic** decisions are long term and influence future actions.
 - ▶ **Tactical** decisions are short term and generally stand independent of other decisions.
 - ▶ The more strategic the decision, the more it is related to the architecture
 - ▶ How much thought and planning do you need to put into the decision?
How many people are involved in the decision?
Does your decision involve a long-term vision or a short-term action?

Design vs Architecture

- ▶ How much effort will it take to construct or change?
 - ▶ Architectural decisions require more effort to construct or change, while design decisions require relatively less.
- ▶ Does it have significant trade-offs?
 - ▶ *Trade-offs* are the pros and cons you evaluate as you are making a decision.
 - ▶ Decisions that involve significant trade-offs require much more time and analysis to make and tend to be more architectural in nature.
 - ▶ Decisions that have less-significant trade-offs can be made quicker, with less analysis, and therefore tend to be more on the design side.

Architectural characteristics

- ▶ **Domain:** *the thing* you are writing software
 - ▶ Its designing occupies much of the effort
- ▶ **Architectural characteristics** are the important parts of the construction process of a system, irrespective of the domain.
 - ▶ They represent its operational capabilities, internal structure decisions, and other necessary characteristics.
- ▶ Architectural characteristics can be defined in **three parts**:
 - ▶ They are nondomain design considerations
 - ▶ They influence architectural structure
 - ▶ They are critical to application success (we should try to pick as *few* architectural characteristics as possible)

Implicit and Explicit

- ▶ **Explicit** architectural characteristics are specified in the requirements for the application.
- ▶ **Implicit** architectural characteristics are factors that influence an architect's decisions but aren't called out in the requirements.
 - ▶ Security is often an implicit architectural characteristic.
 - ▶ You must use your knowledge of the problem domain to uncover these architectural characteristics during the analysis phase.

Process architectural characteristics

- ▶ **Process architectural characteristics** are where the software development process intersects with software architecture.
- ▶ They reflect the decisions about the mechanics of building software.
 - ▶ **Modularity:** The degree to which the software is composed of discrete components.
 - ▶ **Testability:** How complete the system's testing is and how easy these tests are to run, including unit, functional, user acceptance, and exploratory tests.
 - ▶ **Deployability:** How easy and efficient it is to deploy the software system.
 - ▶ **Extensibility:** How easy it is for developers to extend the system.
 - ▶ **Decouple-ability:** This measures the extent to which it is possible to decouple parts in specific ways to achieve certain benefits.

Structural architectural characteristics

- ▶ *Structural architectural characteristics* affect the internal structure of the software system, including factors like the degree of coupling between components and the relationships between different integration points.
 - ▶ **Security**
 - ▶ **Maintainability:** How easy it is for architects and developers to apply changes to enhance the system and/or fix bugs.
 - ▶ **Portability:** How easy it is to run the system on more than one platform.
 - ▶ **Extensibility:** How easy it is for developers to extend the system.
 - ▶ **Localization:** How well the system supports multiple languages, units of measurement, currencies, ...

Operational architectural characteristics

- ▶ *Operational architectural characteristics* represent how architectural decisions influence what operational team members can do.
 - ▶ **Availability:** What percentage of the time the system needs to be available.
 - ▶ **Recoverability:** How quickly the system can get online again and maintain business continuity in case of a disaster.
 - ▶ **Robustness:** The system's ability to handle errors and boundary conditions while running.
 - ▶ **Performance:** How well the system achieves its timing requirements using the available resources.
 - ▶ **Reliability/safety:** Whether the system needs to be fail-safe, or if it is mission critical in a way that affects lives.
 - ▶ **Scalability:** How well the system performs and operates as the number of users or requests increases.

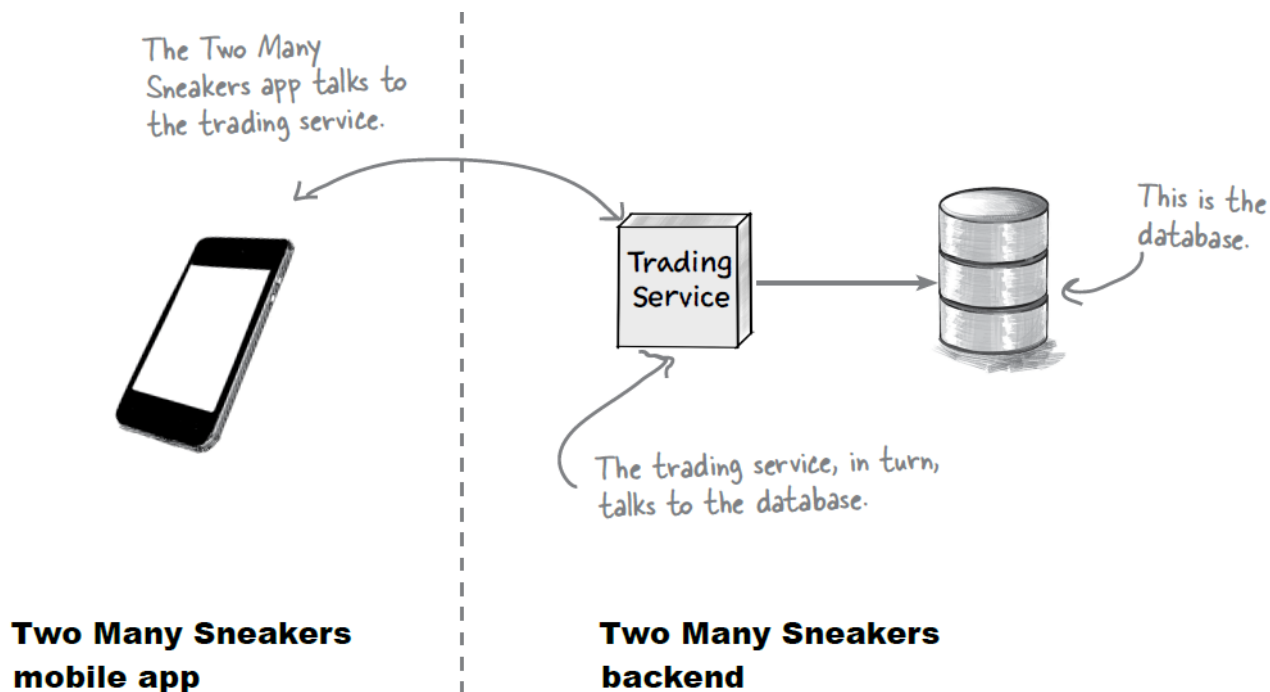
Cross-cutting architectural characteristics

- ▶ **Security:** How secure the system is, holistically.
- ▶ **Legal:** How well the system complies with local laws about data protection, ...
- ▶ **Authentication/authorization:** How well the system ensures users are who they say they are.
- ▶ **Privacy:** How well the system hides and encrypts transactions.
- ▶ **Accessibility:** How easy is it for all your users to access the system.
- ▶ **Usability:** How easy is it for users to achieve their goals.

Architectural characteristics motivations

- ▶ Architectural characteristics don't just appear out of thin air.
 - ▶ **The problem domain**
 - ▶ This means to translate the items stated in requirements documents into their corresponding architectural characteristics
 - ▶ **Environmental awareness**
 - ▶ It is important to understand organizational priorities so we can make more durable decisions.
 - ▶ **Holistic domain knowledge**
 - ▶ This is information that isn't explicitly spelled out in the requirements but that you implicitly understand about important aspects of the domain.

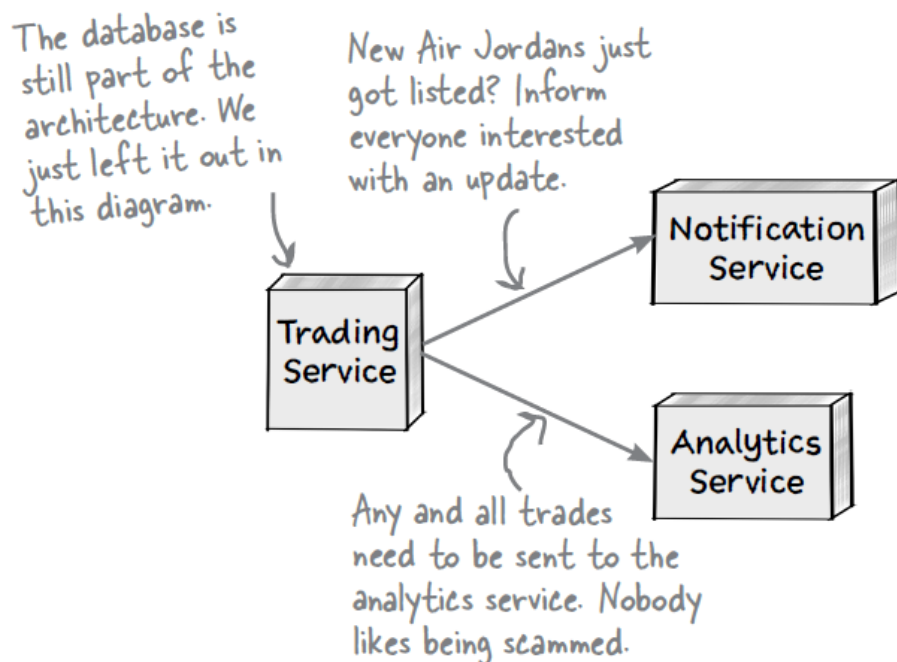
Trade-offs -- example



The Two Many Sneakers app knows to talk to the trading service to fetch and update data (like a photo of a mint-condition pair of Nikes). The trading service fetches data and updates the database.

Trade offs -- example

- ▶ To address security concerns the trading service has to notify the notification and analytics services anytime something of interest happens in the app.

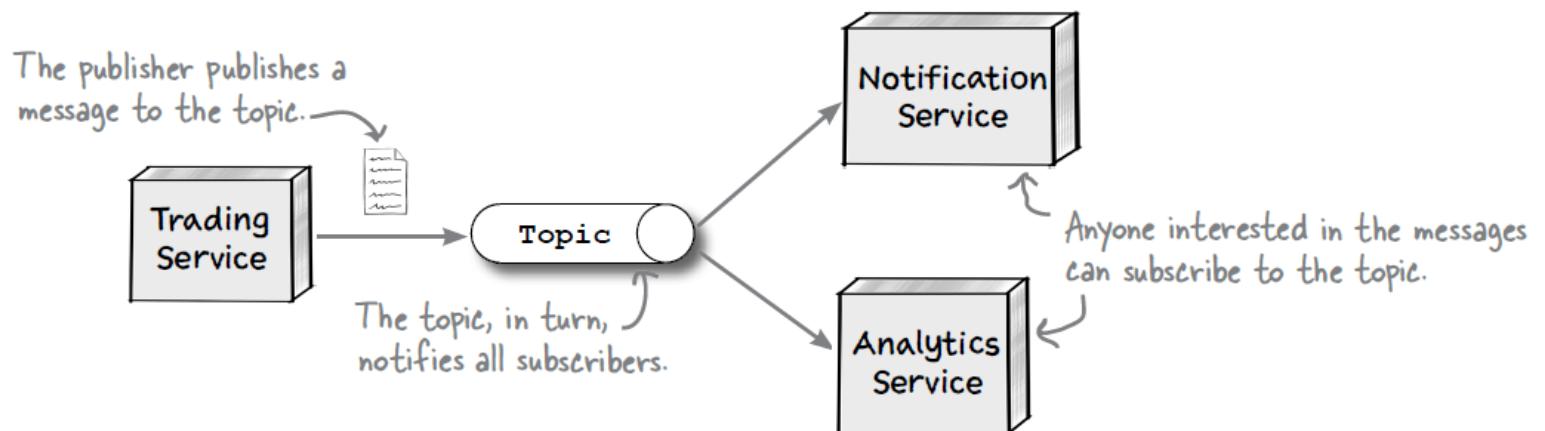
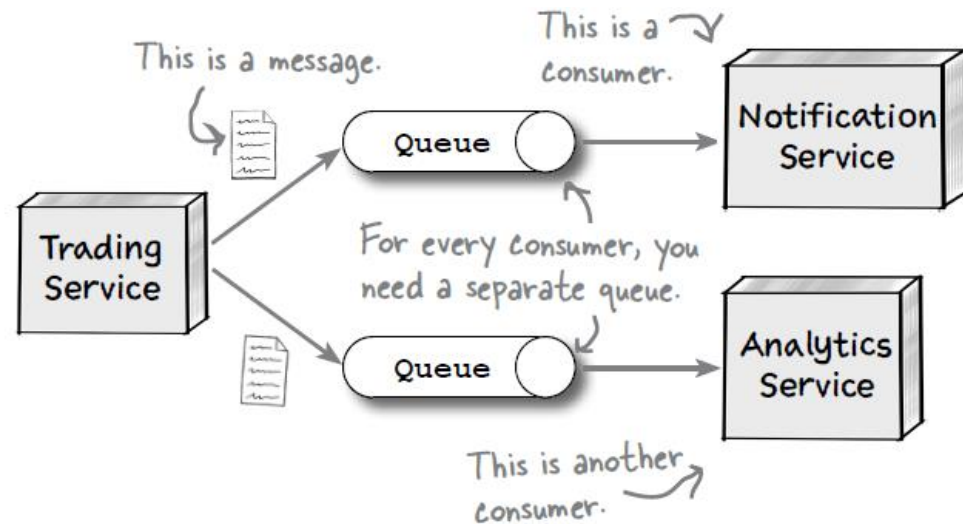


Other services can be added, the architecture is flexible

Nevertheless, we don't know what data to send the notification and analytics services and to the other services

Communicating with downstream services

► Two options



Communicating with downstream services

Heterogeneous is just a fancy way of saying "different."

Using Queues	
Pros	Cons
- Supports heterogeneous messages for different consumers - Allows independent monitoring and scaling (helps scalability) - More secure (improves security)	- Higher degree of coupling (hurts extensibility) - Trading service must connect to multiple queues - Requires additional infrastructure

Whiteboards are great for brainstorming trade-offs with your team.

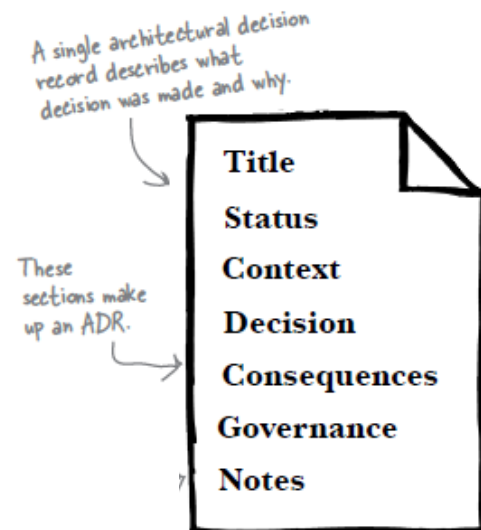
Using Topics	
Pros	Cons
- Low coupling (helps extensibility) - Trading service only has one place to publish messages	- Homogeneous message for all services - Can't monitor or scale a topic independently (hurts scalability) - Less secure (hurts security)

Trade-offs -- example

- ▶ Now, queues or topics?
 - ▶ We have to map the requirements on pros / cons of the two architectures
- ▶ Two main rules:
 - ▶ Everything in software architecture is a trade-off
 - ▶ Architecture Tradeoff Analysis Method
<https://insights.sei.cmu.edu/library/architecture-tradeoff-analysis-method-collection>
 - ▶ Why is more important than how
 - ▶ Let's say the team concludes that you're going to use a cache to improve your application's performance.
 - The result of your analysis is that your system starts using a cache somewhere.
 - The *what* is easy to spot.
 - ▶ That decision is important, but so are the circumstances in which you made the decision, its impact on the team implementing it, and *why*, of all the options available to you, you chose what you did.

Architectural decision records (ADRs)

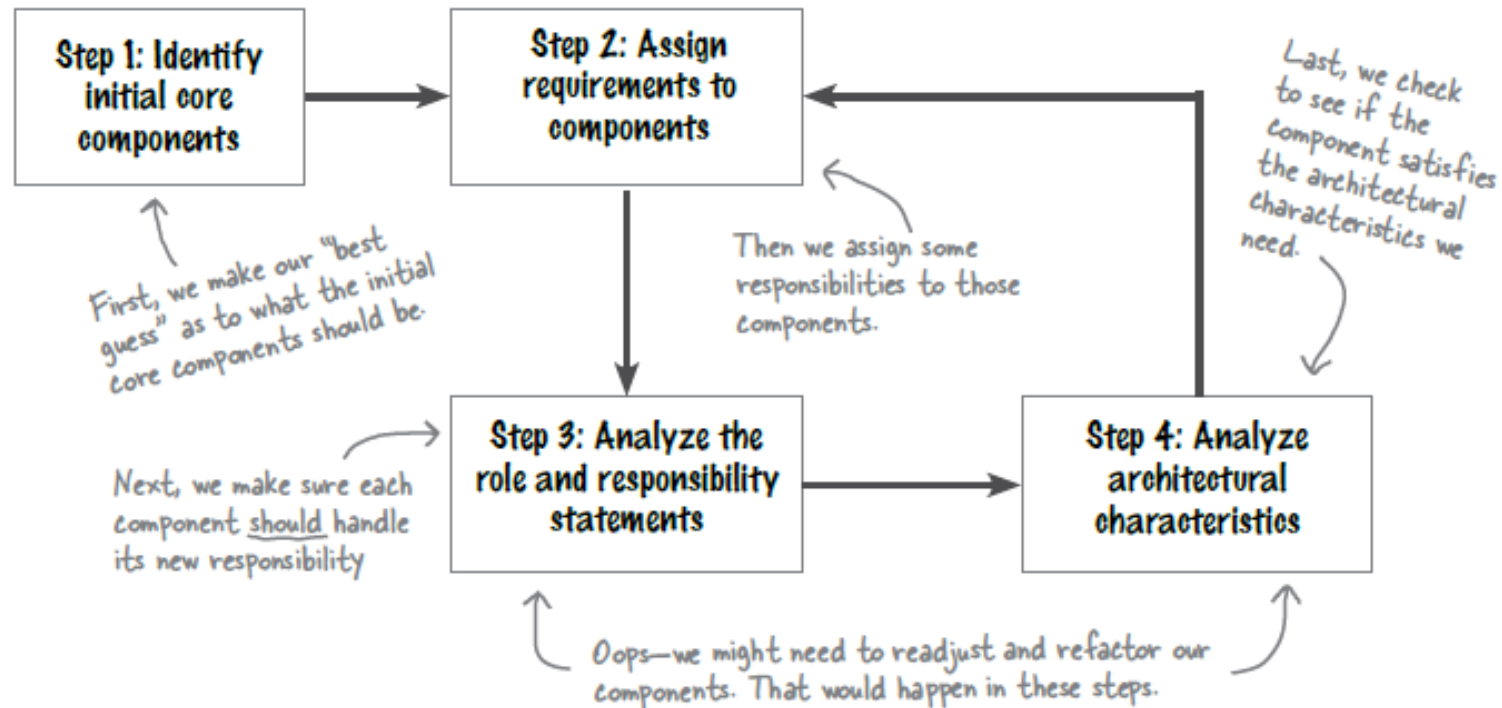
- ▶ An ADR describes a specific architectural decision.
 - ▶ There is an ADR for every architectural decision.
- ▶ The ADRs build up into an ***architectural decision log***.
- ▶ An ADR has seven sections:
 - ▶ Title (A short name and an ordered identified)
 - ▶ Status (Request for Comment, Proposed, Accepted, Superseded),
 - ▶ Context (the circumstances that drove you to make the decision)
 - ▶ Decision,
 - ▶ Consequences (The consequences—good and bad—of the decisions),
 - ▶ Governance (How you ensure that the organization doesn't deviate from the decision),
 - ▶ Notes.



Logical components

- ▶ Logical components are the functional building blocks of a system that make up what is known as the problem domain.
- ▶ A **logical architecture** shows all of the logical building blocks of a system and how they interact with each other (known as **coupling**).
- ▶ A **physical Architecture** shows things like the architectural style, services, protocols, databases, user interfaces, API gateways, ...
- ▶ The logical architecture of a system is independent of the physical architecture.

Creating a logical architecture



Creating a logical architecture

- ▶ Identifying initial core components
 - ▶ The *workflow approach* is an effective way to identify an initial set of core components by thinking about the major workflows of the system.
 - ▶ The *actor/action approach* is particularly helpful if you have multiple actors (users) in the system.
 - ▶ You start by identifying the various actors.
 - ▶ You identify some of the primary actions they might take and assign each action to a new or existing component.
 - ▶ Pay attention to the *entity trap*
 - ▶ A component that has too many responsibilities.
 - ▶ All too often, components in the entity trap become convenient dumping grounds for all functionality related to those entities.

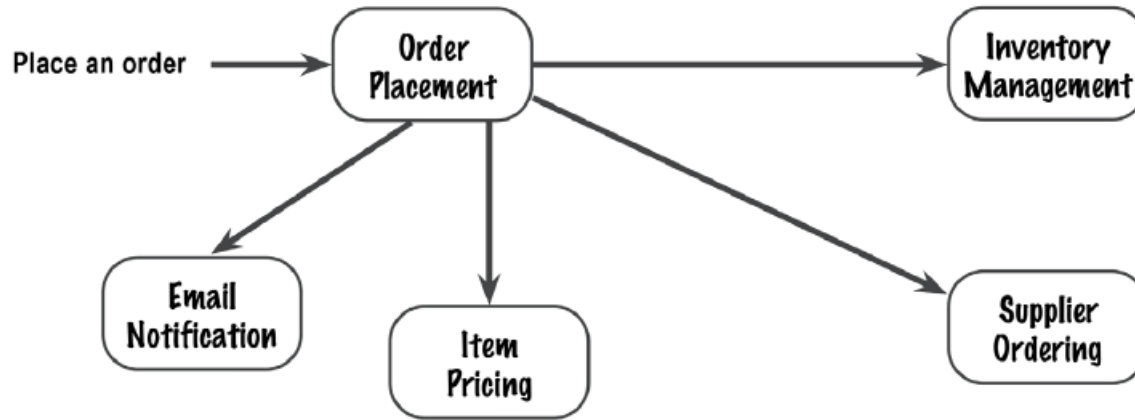
Creating a logical architecture

- ▶ Assigning requirements
 - ▶ You'll take functional user stories or requirements and figure out which component should be responsible for each one
- ▶ Analyze roles and responsibilities
 - ▶ The component to which you are assigning functionality *should* be responsible for the functionality and doesn't end up doing too much.
 - ▶ Check to see if the functionality is closely related.
 - ▶ This is known as ***cohesion***: the degree and manner to which the operations of a component are interrelated.
- ▶ Analyze characteristics
 - ▶ The final step in identifying initial core components is to verify that each component aligns with the ***driving architectural characteristics*** that are critical for success

Coupling

- ▶ **Component coupling:** the degree to which components know about and rely on each other.
 - ▶ The more the components interact, the more tightly coupled the system is and the harder it will be to maintain.
- ▶ **Afferent coupling** is the degree and manner to which other components are dependent on some target component.
 - ▶ It's sometimes referred to as *fan-in*, or *incoming*, coupling.
- ▶ Being dependent on others is known as **efferent coupling**.
 - ▶ Efferent coupling is the opposite of afferent coupling: the number of components on which a target component depends.
 - ▶ It's also known as *fan-out coupling* or *outgoing coupling*.

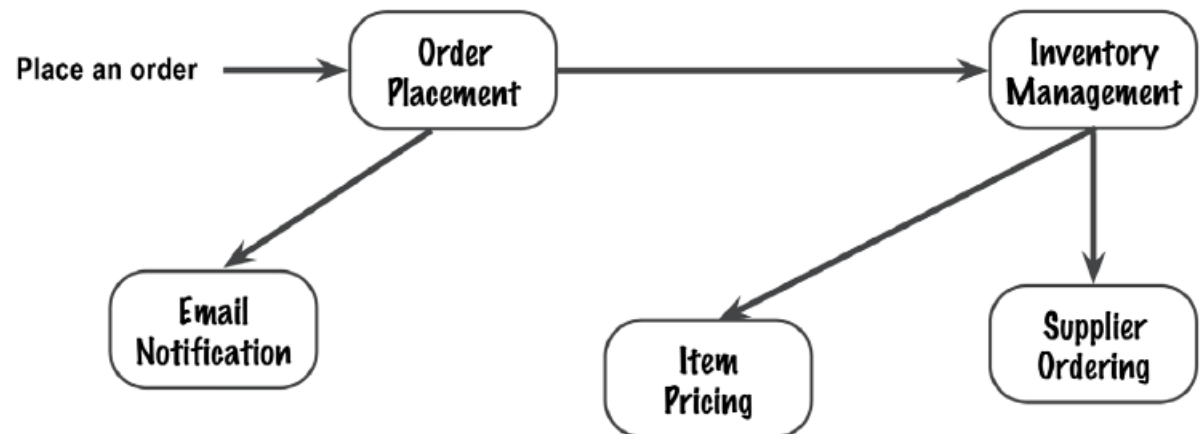
Tightly coupled



Example

- ▶ Centralized workflow, but more risk with each change

Loosely coupled



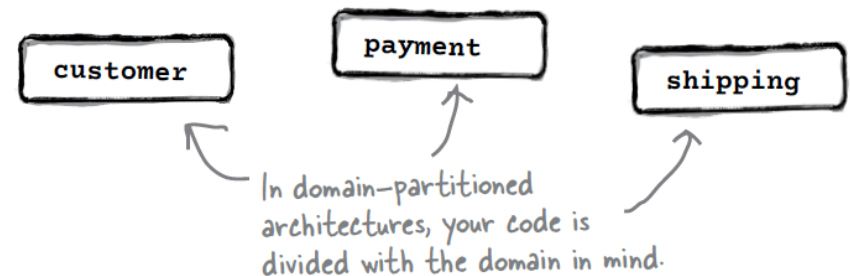
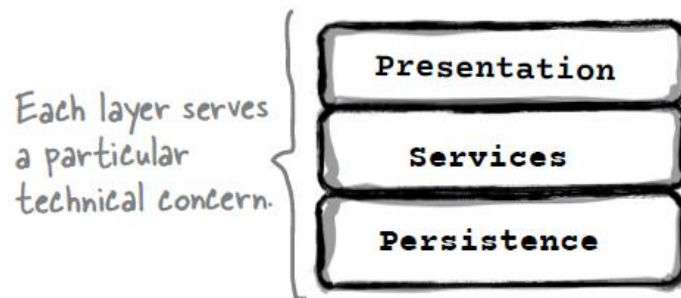
- ▶ Distributed workflow, but less risk with each change

Architectural Styles

		Partitioning	
		Technical	Domain
Deployment model	Monolith	Layered Microkernel	Modular monolith
	Distributed	Event-driven	Microservices

Partitioning: Technical versus domain

- ▶ In a ***technically partitioned architecture***, code is separated by technical concerns in horizontal layers.
 - ▶ presentation tier,
 - ▶ business (services) layer, and so on.
- ▶ In ***domain-partitioned architectures***, the structure of the system is aligned with the domain.



Deployment model: Monolithic versus distributed

- ▶ In a ***monolithic architecture***, you deploy all the logical components that make up your application as one unit.
 - ▶ This also means that your entire applications runs as one process.
- ▶ In a ***distributed architecture***, you split up the logical components across many (usually smaller) units.
 - ▶ These units each run in their own process and communicate with each other over the network.

Monolithic deployment models: The pros

- ▶ **Simplicity:** monolithic applications have a single codebase, which makes them easier to develop and to understand.
- ▶ **Cost:** monoliths are cheaper to build and operate because they tend to be simpler and require less infrastructure.
- ▶ **Feasibility:** monoliths are simple and relatively cheap, freeing you to experiment and deliver systems faster.
- ▶ **Debuggability:** if you spot a bug or get an error stack trace, debugging is easy, since all the code is in one place.
- ▶ **Reliability:** a monolith is an island. It makes few or no network calls, which usually means more reliable applications.

Monolithic deployment models: The cons

- ▶ **Scalability:** if you ever need to scale one part of the application independently of the others, well, you're in trouble. It's all or nothing with monoliths.
- ▶ **Evolvability:** as monolithic applications grow, making changes becomes harder. Furthermore, since the whole application is one codebase, you can't adapt different technology stacks to different domains if you need to.
- ▶ **Reliability:** because monolithic applications deploy as a single unit, any bug that degrades the service will affect the whole monolith.
- ▶ **Deployability:** implementing any change will require redeploying the whole application, which could introduce a lot of risk.

Distributed deployment models: The pros

- ▶ **Scalability:** Distributed architectures deploy different logical components separately from one another.
- ▶ **Modularity:** Distributed architectures encourage a high degree of modularity because their logical components must be loosely coupled.
- ▶ **Testability:** Each deployment only serves a select group of logical components. This makes testing a lot easier—even as the application grows.
- ▶ **Deployability:** Distributed architectures encourage lots of small units. They evolved after modern engineering principles like continuous integration, continuous deployments, and automated testing became the norm.
- ▶ **Fault tolerance:** Even if one piece of the system fails, the rest of the system can continue functioning.

Distributed deployment models: The cons

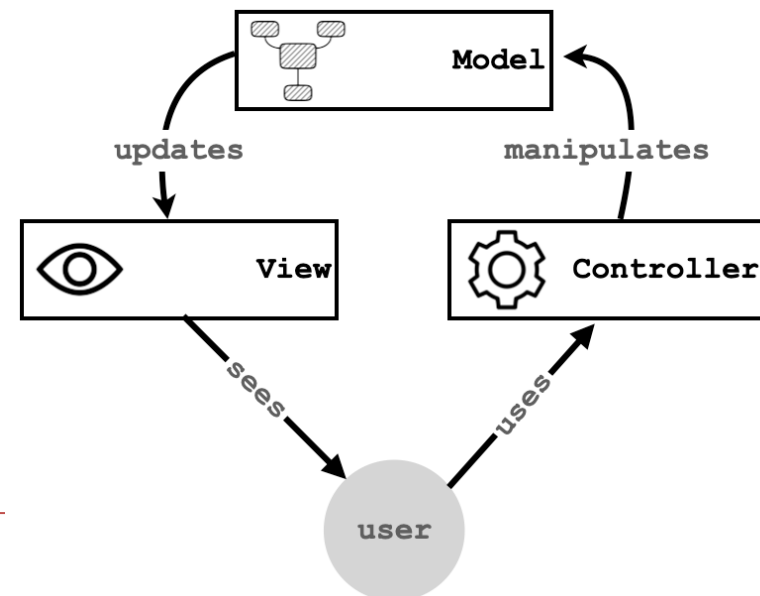
- ▶ **Performance:** Distributed architectures involve lots of small services that communicate with each other over the network to do their work. This can affect performance, and although there are ways to improve this, it's certainly something you should keep in mind.
- ▶ **Cost:** Deploying multiple units means more servers. Not to mention, these services need to talk to one another—which entails setting up and maintaining network infrastructure.
- ▶ **Simplicity:** Distributed systems are the *opposite* of simple. Everything from understanding how they work to debugging errors becomes challenging.
- ▶ **Debuggability:** Errors could happen in any service involved in servicing a request. Since logical components are deployed in separate units, tracing errors can get very tricky.

Layered Architecture

- ▶ We want to implement a website for a restaurant
- ▶ Requirements
 - ▶ **Time to market:** The restaurant is already open. The faster they can get the site online the faster they can start making money.
 - ▶ **Separation of responsibilities:** The company has part-time help with specialized skills, such as user interface (UI) specialists and database administrators (DBAs). Thus, it would help to keep each part of the system separate.
 - ▶ **Extensible:** we find ways to extend and reuse parts of the system if the company grows.

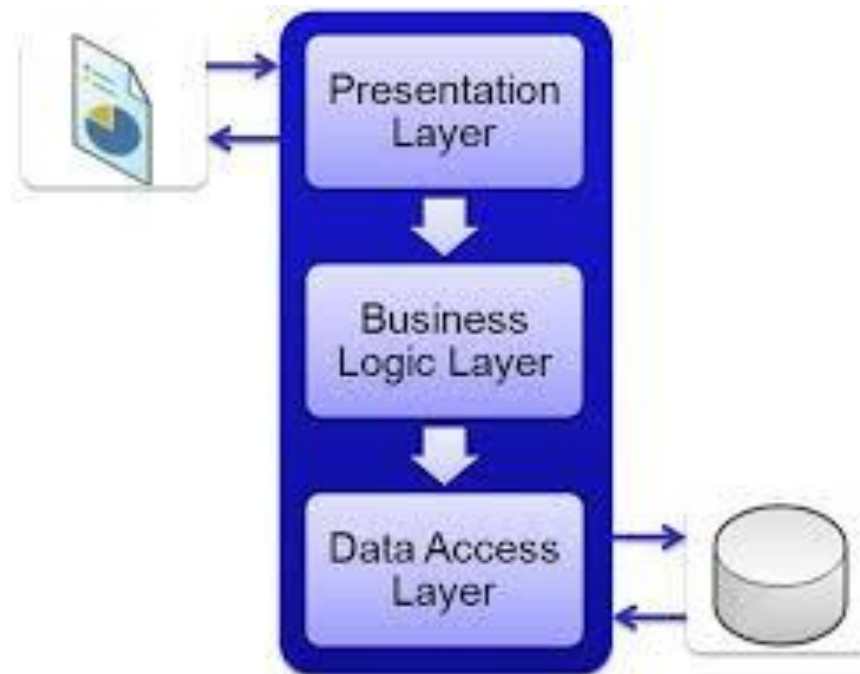
Model-View-Controller

- ▶ The **Model-View-Controller** (MVC) is a design pattern used in the design of web applications.
- ▶ It separates capabilities based on their purpose.
 - ▶ The **model** represents business logic and entities in the application;
 - ▶ the **view** represents the user interface;
 - ▶ the **controller** handles the workflow, stitching model elements together to provide the application's functionality.



Layering MVC

- ▶ Design patterns represent logical solutions to problems, but architecture must deal with real-world constraints like databases, user interfaces, and other implementation details.



The layered architecture is a *technically partitioned* architectural style, typically deployed as a *monolith*.

Layers

- ▶ Layers are created with packages or namespaces, just like domain components.
- ▶ To maintain the separation of concerns, the layers' package structures typically reflect their place within the partitioning:
 - ▶ `com.orderapp.presentation`
 - ▶ `com.orderapp.workflow`
 - ▶ `com.orderapp.persistence`

Layers

- ▶ The ***presentation layer*** is responsible for interacting with the user, serving the same purpose as the *view* part of MVC.

```
def UI_layer(request):  
    data = request.get_data()  
    return business_logic_layer(data)
```

- ▶ The ***workflow layer*** is responsible for processing each request from the UI layer and returning a response.

```
def business_logic_layer(data):  
    processed_data = process_data(data)  
    return data_access_layer(processed_data)
```

- ▶ The ***persistence layer*** is responsible for accessing the data from the database and returning it to the workflow layer.

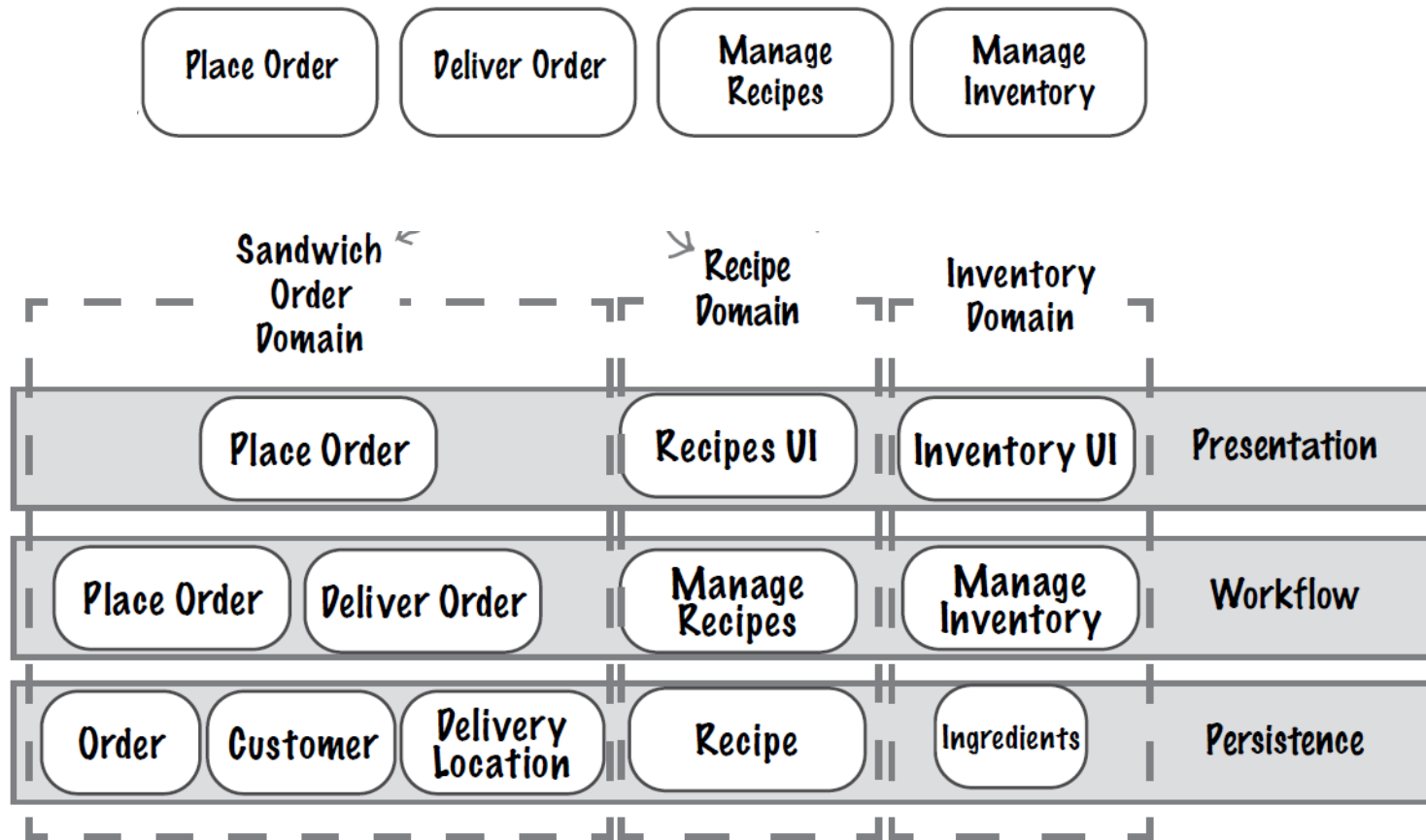
```
def data_access_layer(data):  
    retrieved_data = retrieve_data(data)  
    return retrieved_data
```

Behavior of the application

- ▶ The domain behavior lives *across* the layers in this architecture.
 - ▶ The *domain* represents logical components based on the problem you're trying to solve.
- ▶ The layers in this architecture represent *technical* capabilities
- ▶ The domain maps onto the layered architecture, sometimes spreading between layers.

Domains, components, and layers

► Components

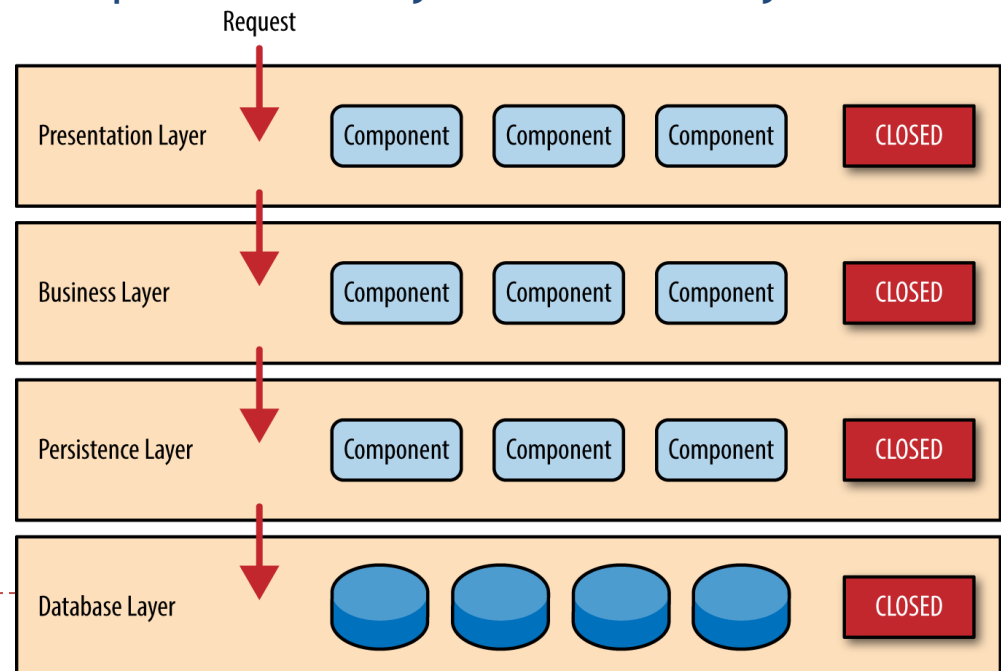


Layered Architecture

- ▶ The layered architecture is a *technically partitioned* architecture (as opposed to a *domain-partitioned* architecture).
- ▶ Components are grouped by their technical role in the architecture.
 - ▶ Any business domain is spread throughout all of the layers.
- ▶ **Layers are isolates:** changes made in one layer don't impact or affect components in other layers.
 - ▶ Each layer is independent of the other layers, having no knowledge of the inner workings of other layers.
 - ▶ To support layers of isolation, layers involved have to be closed.
 - ▶ If the presentation layer can directly access the persistence layer, then changes made to the persistence layer would impact both the business layer *and* the presentation layer.
- ▶ The layers of isolation concept allows any layer to be replaced without impacting any other layer.

Isolation

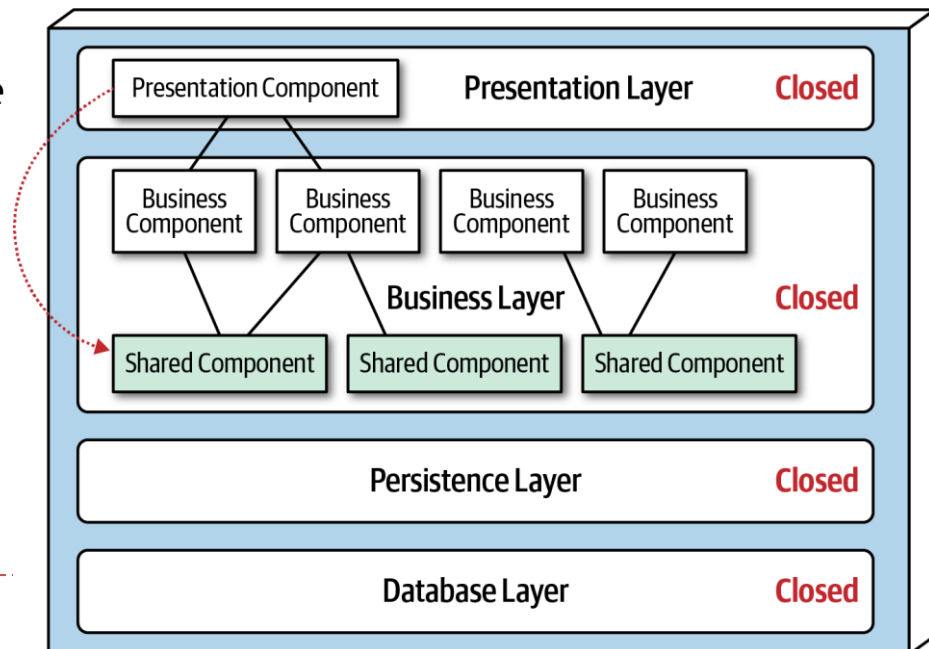
- ▶ To preserve isolation layers, in the architecture are marked as being *closed*.
 - ▶ A closed layer means that as a request moves from layer to layer, it must go through the layer right below it to get to the next layer below that one.
 - ▶ A request originating from the presentation layer must first go through the business layer and then to the persistence layer before finally hitting the database layer.



Layered Architecture

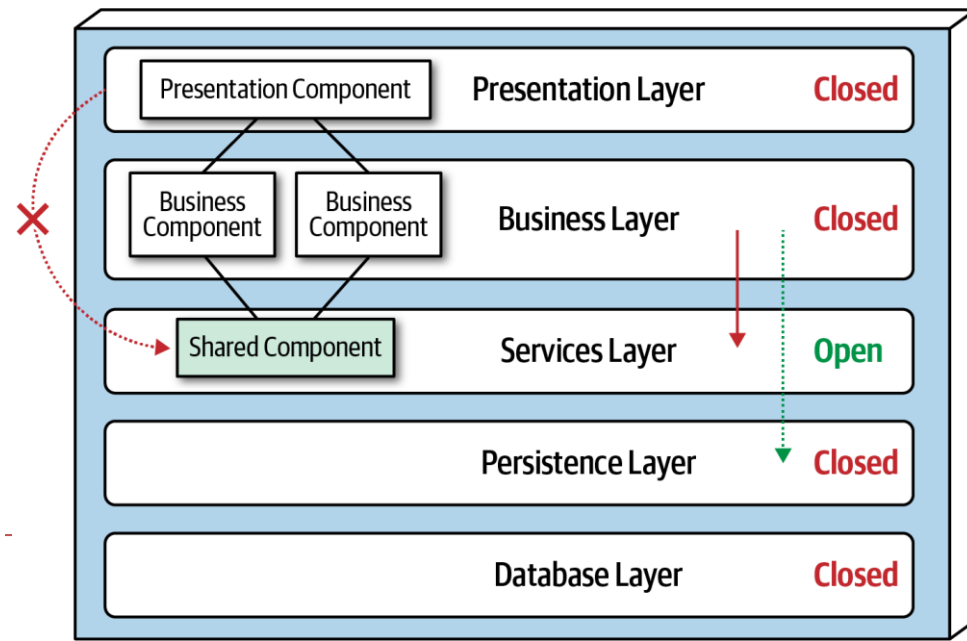
► Adding Layers

- There are times when it makes sense for certain layers to be open.
 - Suppose there are shared objects within the business layer with common functionality for business components (such as date and string utility classes, logging classes, ...).
 - There is an architecture decision stating that the presentation layer is restricted from using these shared business objects.
- This scenario is difficult to govern and control because architecturally the presentation layer has access to the business layer, and hence has access to the shared objects within that layer.



Layered Architecture

- ▶ One way to architecturally mandate this restriction is to add to the architecture a new services layer containing all of the shared business objects.
- ▶ Adding this new layer now architecturally restricts the presentation layer from accessing the shared business objects because the business layer is closed.
- ▶ The new services layer must be marked as *open*; otherwise the business layer would be forced to go through the services layer to access the persistence layer.



Drivers for layered architecture

- ▶ **Specialization**

- ▶ A layered architecture allows organizations to split teams into specialists, sharing their capabilities between different projects

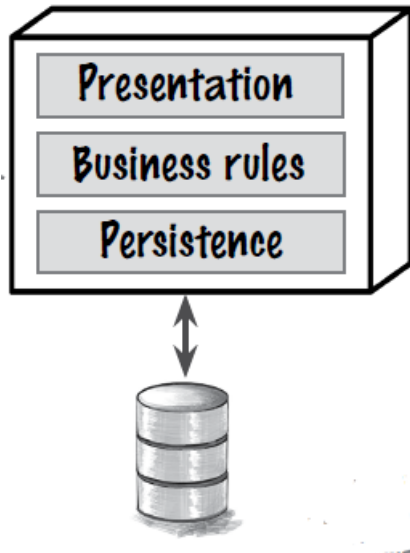
- ▶ **Ease of (technical) reuse**

- ▶ For example, if all persistence code resides in a single layer, it's easier for developers to find, update, and reuse it.

- ▶ **Matches physical separation**

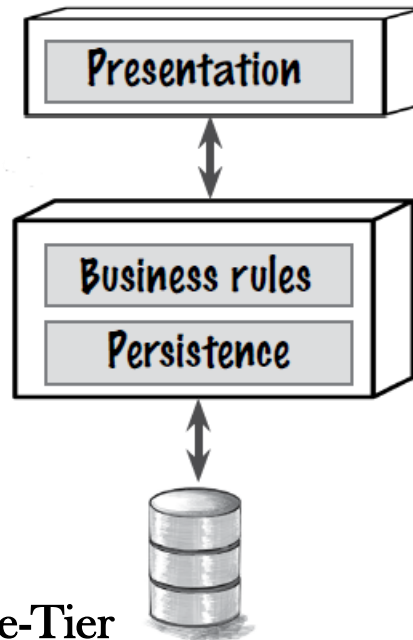
- ▶ It's common for teams to implement different layers in different technology stacks (such as JavaScript, Java, and MySQL).

Physical architectures



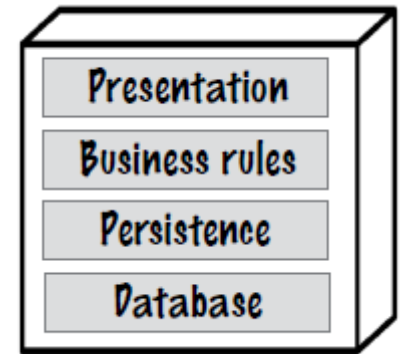
Two-Tier

- + Rich user interface
- + High performance
- + Simple
- Medium scalability
- Becomes complex when it gets big
- Medium reliability



Three-Tier

- + Detached UI (web)
- + Highest scalability
- + Distributed architecture benefits
- Least reliability
- More complex
- Distributed architecture headaches



Embedded/Mobile

- + Self-contained
- + Single tech stack
- + Highly tunable to hardware devices
- Least scalable
- Resource- constrained
- Often tied to implementation platform

Layered architecture pros

- ▶ **Feasibility.** If time and budget are important, the simplicity of this architecture is quite appealing.
- ▶ **Performance.** Well-designed layered monoliths demonstrate high performance: no need for network calls that could decrease performance.
- ▶ **Technical partitioning.** If several teams need the same data functionality, they could implement it once in a persistence layer and then share it across teams.
- ▶ **Quick to build.** Simplicity plus a single work/deployment unit means that teams can build small systems quite rapidly.
- ▶ **Data-intensive.** Systems that do a lot of data-level processing may benefit from a layered architecture because it isolates data processing in a single database that's optimized for the task.

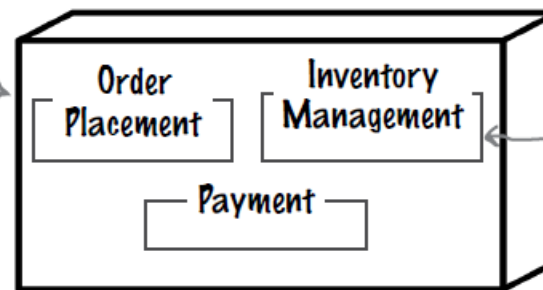
Layered architecture cons

- ▶ **Scalability.** Monoliths generally become constrained by some resource (memory bandwidth, and so on).
- ▶ **Deployability.** As monolithic systems get bigger, deployments tend to become more complex.
- ▶ **Testability.** High coupling and a large codebase make testing harder and harder over time.
- ▶ **Big ball of mud.** Because everything is connected to everything else, this architecture can become a highly coupled mess without careful governance.

Modular monoliths

- ▶ You partition the application by *functionality* and not by technical concerns.
- ▶ Every business operates within a certain domain—like banking, education, or retail.
- ▶ Applications can have several *subdomains*, like Order Placement, Payment, and Inventory Management.
- ▶ Together, they make up the Application domain.
- ▶ You organize your application according to these subdomains, separating them into modules (e.g., packages, namespaces).

Modular monolith
is also a monolithic
deployment model.



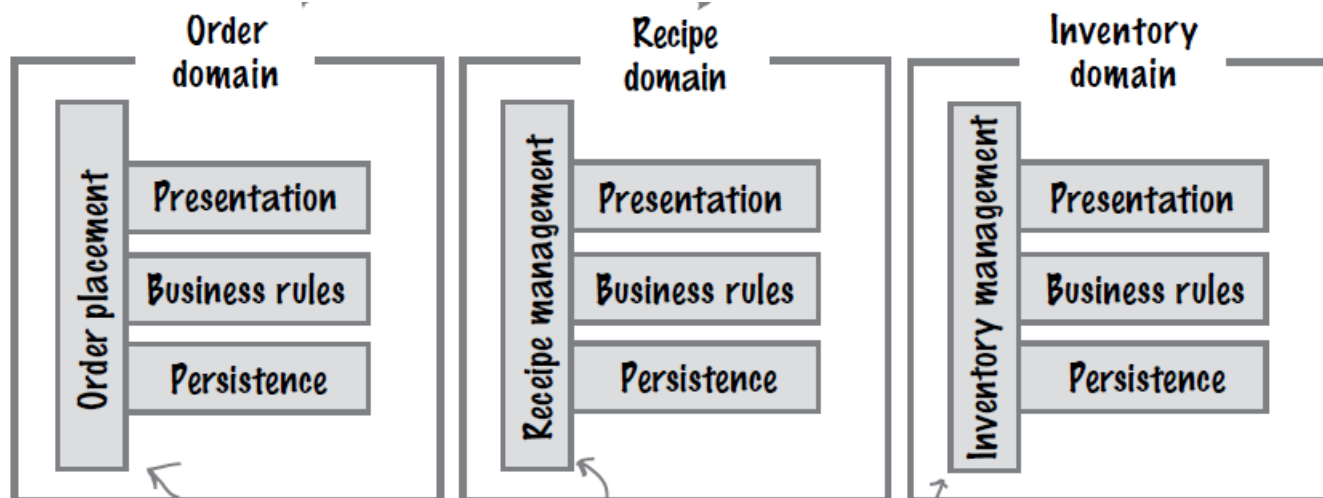
Rather than organizing
the application by
technical concerns,
we organize them by
business domain.

Modular monoliths
also use a monolithic
database.



Modular monoliths

- ▶ You organize the application in vertical slices scoped by business concern.
 - ▶ Each vertical slice aligns with a piece of the domain and is encapsulated in a module.
 - ▶ Every module contains a set of business functions—for example, order placement, order completion, and order delivery would all be part of the Order Placement module
- ▶ Changes to the domain that affect many layers require coordination between different teams. You need to ensure that everybody's changes work with everybody else's.
- ▶ Rather than having teams that specialize in some layer, you have *cross-functional teams*, each specializing in a *domain*.

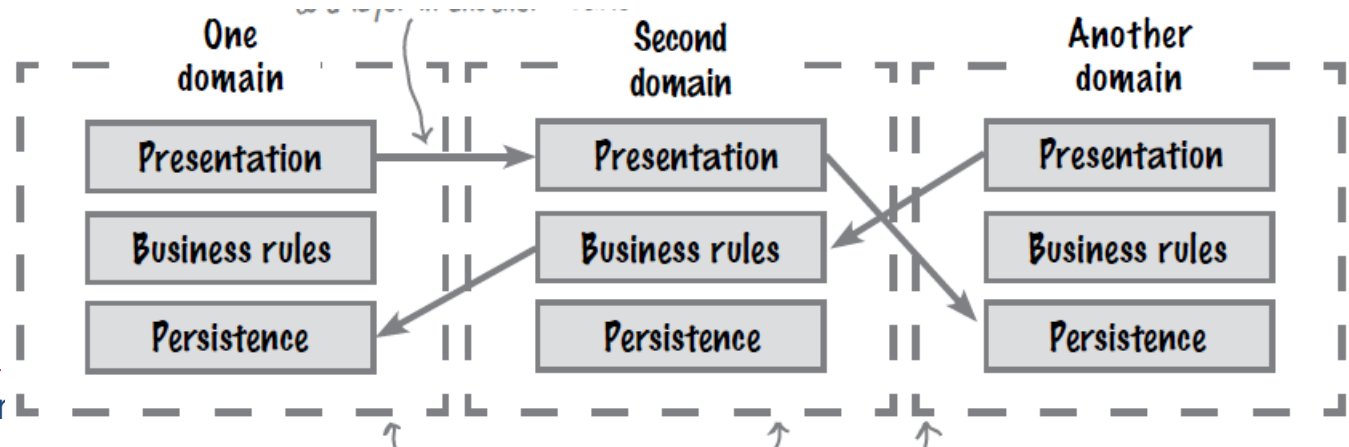


Modular monoliths

- ▶ The application is organized by domain even if each module consists of layers.
 - ▶ The reason is that the system still needs to process and respond to requests.
 - ▶ So you need an entry point (presentation layer), some business processing (workflow layer), and maybe a data store to write to (persistence layer).
 - ▶ It makes sense to separate the responsibilities, as the layered architecture.
- ▶ The application is divided into subdomains.
 - ▶ The fact that each module is composed of a bunch of technically partitioned layers is an implementation detail, as opposed to an architectural concern.
- ▶ In other words, ***how a module is laid out internally isn't how the architecture is partitioned.***
- ▶ Modular monoliths are domain-partitioned.

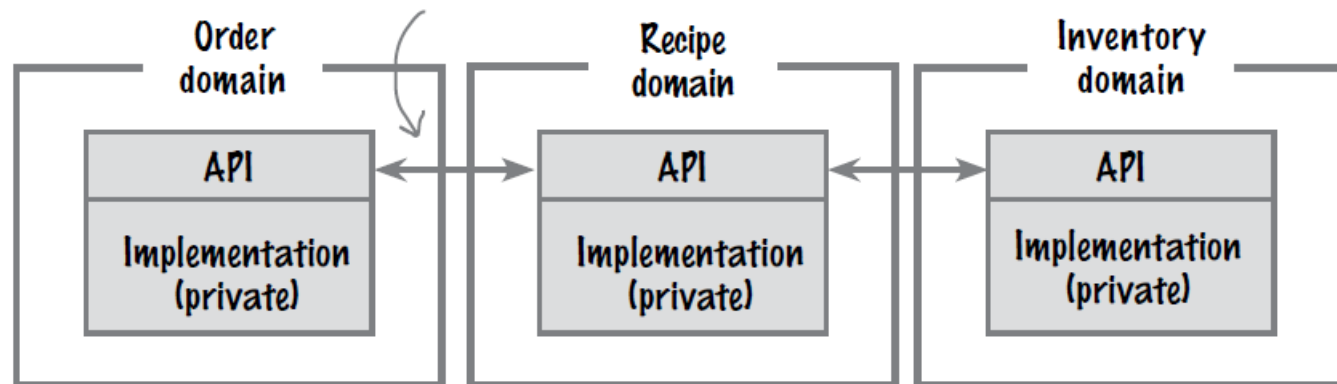
Keeping modules modular

- ▶ Modular monoliths are generally contained in one codebase.
- ▶ That makes it easy for someone working in one module to inadvertently reach into another module and end up coupling the two modules together.
- ▶ The *philosophy* of the modular monolith centers on ***partitioning by domain within a monolithic deployment model***.
- ▶ Your objective should be to create loosely coupled modules so that changing one doesn't affect others.



Keeping modules modular

- ▶ From a design perspective, we have to *think of each module as a separate service*.
 - ▶ They aren't *really* separate—they all still constitute one monolithic deployment.
- ▶ Each “service” exposes a public API while shielding its internal implementation from the other modules.
- ▶ As long as modules talk to one another through their public APIs, you can safely change one module without affecting others—thereby reducing their coupling.

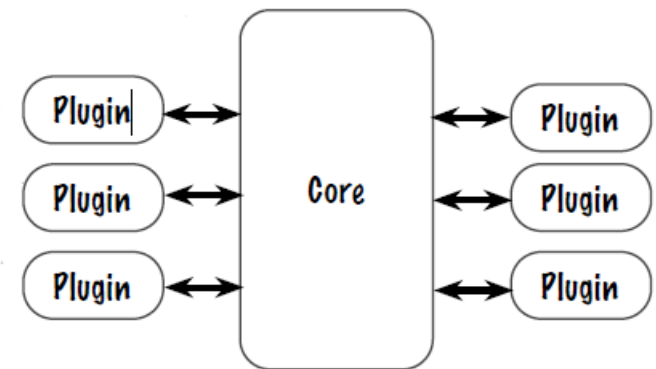


Keeping modules modular

- ▶ Modules sharing the same database can be coupled at the data level.
 - ▶ Modularizing the code may not be enough!
- ▶ Most developers are not used to thinking vertically along business concern lines or breaking up their code into separate modules.
 - ▶ Extending that modularity all the way to the database sounds like a great idea—*it is*—but it may be too much to take on all at once.
 - ▶ Feel free to evolve your architecture over time when needed, rather than trying to get it all right the first time around.
- ▶ Do your architectural characteristics push you to pick the modular monolith architecture? If so, start by modularizing your code first.
 - ▶ Once your team gets the hang of thinking modularly, then see if it helps to take that approach all the way into the database.

Microkernel Architecture

- ▶ The **microkernel** architecture derives its name from operating system design.
 - ▶ The **kernel**, or core, of an operating system is very small, offering only the most basic capabilities.
- ▶ A microkernel architecture consists of two primary parts: its *core* and its *plugins* (any number of them)
- ▶ The core in a microkernel:
 - ▶ provides minimal functionality
 - ▶ rarely changes after its initial implementation
 - ▶ does not provide custom behavior
- ▶ Plugins in a microkernel:
 - ▶ provide all custom behavior
 - ▶ work in isolation from one another
 - ▶ are deployed independently



Encapsulated versus distributed plugins

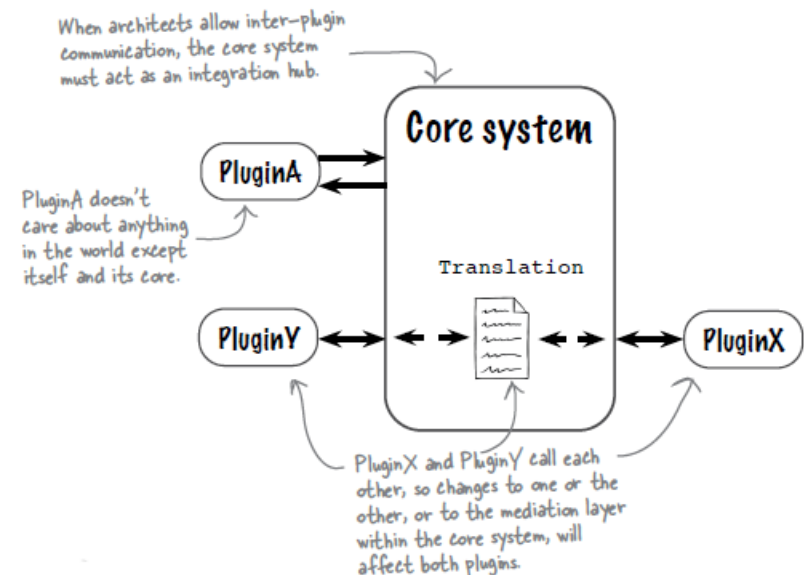
- ▶ The core system in a microkernel is where the plugins plug in.
- ▶ Generally that connection is implemented via an *interface*.
 - ▶ The plugin implements the interface
 - ▶ The core system supports that component via that interface.
- ▶ If we design a microkernel as a monolithic architecture:
 - ▶ Plugins use the same technology stack as the core and are deployed as native components for the platform (like JAR files for Java, DLLs for .NET).
- ▶ If the architecture is distributed, plugins can be
 - ▶ web endpoints, event queues, ...
 - ▶ called synchronously or asynchronously.
 - ▶ And developers aren't restricted to the core's implementation platform.

Plugin contracts

- ▶ Plugins typically communicate with the core system by implementing an interface that the core supports.
 - ▶ That makes it possible for plugins to communicate *with each other* “through” the core system.
- ▶ Allowing inter-plugin communication can have some serious negative trade-offs.
 - ▶ First, it requires consistent ***contracts*** between the core and the plugins, which eventually involves versioning.
 - ▶ Second, dependencies between plugins create availability issues, because you must guarantee that all necessary plugins are present at runtime.

Plugin contracts

- ▶ When architects implement microkernel architectures, we usually ensure that the core calls plugins using a **contract** (another word for *interface*).
- ▶ That communication is *solely between the plugin and the core*, not between plugins.
- ▶ If you allow communication between plugins, the core has to act as their intermediary.



Microkernel architecture details

▶ Registry

- ▶ It allows the core system to know about plugins available and how to get to them.
- ▶ It contains information about each plugin module, including things like its name, data contract, and remote access protocol details.
- ▶ The registry can be as simple as an internal map structure owned by the core system containing a key and the plug-in component reference.

▶ Contracts between the plug-in components and the core system

- ▶ Are usually standard across a domain of plug-in components
- ▶ Include behavior, input data, and output data returned from the plug-in component.
- ▶ Custom contracts for plug-in components developed by a third party.
 - ▶ It is common to create an adapter between the plug-in contract and your standard contract.

Example

registry

```
Map<String, String> pluginRegistry = new HashMap<String, String>();
static {
    pluginRegistry.put(HEADER, "ValidatorHeaderPlugin");
    pluginRegistry.put(SQL, "ValidatorSqlPlugin");
    pluginRegistry.put(INTERCEPT, "ValidatorInterceptPlugin");
    pluginRegistry.put(AUDIT, "ValidatorAuditPlugin");
}
```

plug-in contracts

```
public interface ValidatorPlugin {
    public ValidatorOutputData execute(ValidatorInputData data);
}
```

```
public class ValidatorInputData {
    public String moduleName;
    public List<String> moduleContents;
}
```

```
public class ValidatorOutputData {
    public String validationResults;
}
```

Microkernel architecture

core system

```
private String executeChecks(String moduleName) throws Exception {  
    for (Map.Entry<String, String> entry : pluginRegistry.entrySet()) {  
        Class<?> c = Class.forName(PLUGIN_PKG + entry.getValue());  
        Constructor<?> con = c.getConstructor();  
        ValidatorPlugin plugin = (ValidatorPlugin)con.newInstance();  
        outputData = plugin.execute(inputData);  
        System.out.println(outputData.validationResults);  
    }  
}
```

Microkernel pros

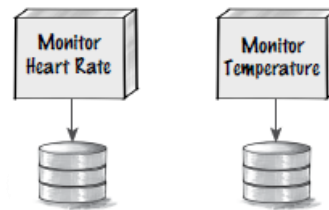
- ▶ **Custom behavior**
- ▶ ***Adaptability***, i.e., implies the ability to keep existing functionalities and continue adding more.
- ▶ ***Evolvability***, means that architects can make fundamental underlying changes that gradually evolve the system away from its old behaviors.
- ▶ **Simple structure** A microkernel has two basic moving parts (the core and plugins), making it easy for developers to understand and implement
- ▶ **Partitioning**

Microkernel cons

- ▶ **Misaligned volatility:** the core system should change very little once it is implemented. If it changes a lot, that is a sign that this is not the ideal architectural solution.
- ▶ **Sharing between plugins:** sharing dependencies (such as shared libraries) between plugins is generally a bad idea because it creates headaches around coupling and deployment.
- ▶ **Chatty plugins:** allowing plugins to interact may be tempting, but it comes with a host of difficult trade-offs. Successful systems that use this approach (for example, Eclipse) do so at the cost of high complexity.
- ▶ **Performance:** when using distributed plugins for a physical architecture, you might notice an impact on performance, depending on the communication protocols and how much information passes between the core and the plugins.

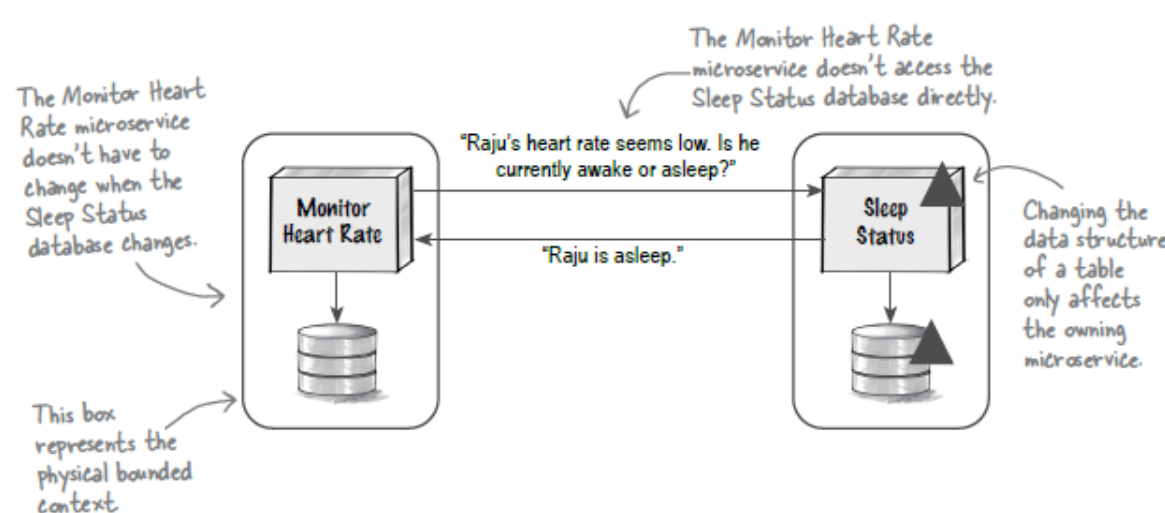
Microservices

- ▶ A **service** is a separately deployed unit of software that performs some business or infrastructure process.
 - ▶ For example, a service in an application for monitoring the health of a patient might perform a lot of functions, including monitoring the patient's heart rate, blood pressure, temperature, ...
- ▶ The prefix **micro-** in microservice refers not to physical size, but to *what the service does*.
 - ▶ For example, a Monitor Heart Rate service is single-purpose and does one thing really well—it monitors a heart rate.
- ▶ A microservice is a single-purpose, separately deployed unit of software that does one thing really well.



Microservices

- ▶ Microservices own their own data.
- ▶ The primary reason is to *manage change control*.
- ▶ Physical bounded contexts help manage change and coupling.
- ▶ If other microservices need access to data they don't own, they must ask the owning service for it.



Granularity

- ▶ ***A microservice is a single-purpose service that's separately deployed.***
 - ▶ How do you determine the scope of that single purpose?
 - ▶ As an example, consider a functionality for measuring blood pressure.
 - ▶ This means capturing the input from the medical device, recording the input, analyzing the measurements, and alerting a doctor if something is wrong.
- ▶ We can model this functionality in one of three ways:
 - ▶ **Option 1:** Create a single monitoring microservice that performs all monitoring and alerting functions.
 - ▶ **Option 2:** Create two separate microservices—one that captures and records vital signs data, and one that analyzes the data and alerts staff if necessary.
 - ▶ **Option 3:** Create separate microservices, each one performing a specific blood pressure monitoring function.

Granularity

- ▶ Microservices that are too fine-grained tend to communicate more with each other to complete business functions, leading to high levels of coupling, poor performance, and overall reliability issues.
 - ▶ This is commonly referred to as the *Grains of Sand* antipattern, in which services are so small that they start to resemble sand on a beach.
- ▶ Microservices that are too *coarse*-grained are harder and more expensive to maintain, test, and scale (which defeats the whole purpose of using microservices).
 - ▶ **Granularity disintegrators** are forces that tell you to make your service smaller (meaning it's doing less work),
 - ▶ **Granularity integrators** are forces that tell you to make the service bigger (meaning it's doing more work).

Granularity disintegrators

- ▶ **Cohesion.** A single-purpose microservice has functionality that is highly *cohesive*.
 - ▶ If the functionalities lack cohesion, we can break that microservice apart.
- ▶ **Code volatility.** Does one part of the microservice change faster than others?
 - ▶ Constant changes to one part of a microservice mean you have to test the *entire* microservice, including functionalities you didn't change.
 - ▶ Moving a changing function into its own separate microservice isolates those changes.
- ▶ **Fault tolerance and Availability.** Do certain functions frequently produce fatal errors?
 - ▶ If the faulty functionality is in its own separate microservice, it won't affect other functions.
- ▶ **Scalability and Throughput.** Do some parts of the microservice need more scalability?
 - ▶ Breaking the service allows better control over which portions need to scale and which do not.
- ▶ **Access control** The larger the service, the more difficult it is to control access to sensitive information.
 - ▶ Moving sensitive functionalities into their own microservices *isolates* them.

Granularity integrators

▶ Database transactions.

- ▶ When requests involve multiple microservices, you can't perform a single database commit or rollback for all updates.
- ▶ If ***data consistency and integrity*** are important, operations have to take place in a single database transaction combining the functionality into a single microservice.

▶ Workflow and Choreography.

- ▶ Coupling is when a single business request requires separate microservices to communicate with each other.
- ▶ Coupling has negative effects on the system (***performance, scalability, fault tolerance***)
- ▶ If the workflow involves a lot of coordination between the microservices and these characteristics are important, consider combining them.

▶ Data dependencies.

- ▶ When you break a microservice apart, you also have to break its *data* apart.
- ▶ However, if the data is ***highly coupled***, it will be difficult to break it apart and form new physical bounded contexts.
- ▶ If your data is highly coupled and functions in the microservice need to share that data, it makes sense to keep the microservice large and combine the functions.

Sharing functionalities

- ▶ **Code reuse** is a necessary part of software development.
 - ▶ Without it, you would have duplicate functionality almost everywhere in your system.
 - ▶ Functions like logging, metrics streaming, user authorization, and basic utilities like transforming date formats are common in most (if not all) systems.
- ▶ In monolithic systems, you write the common functionality once and use it everywhere in the system, because it's all compiled together as one unit.
- ▶ In *distributed* architectures, each microservice is a separately deployed unit of software.
 - ▶ Common functionality go into either a *shared library* or a *shared service*.

Code reuse with a shared service

- ▶ A ***shared service*** is a separate microservice that contains a shared functionality that other microservices can call remotely.
- ▶ Pros
 - ▶ Changing common code in a shared service doesn't require changing other microservices.
 - ▶ The shared service can be written in any language and on any platform.
- ▶ Cons
 - ▶ Changing a shared service is risky because it can immediately affect other microservices that call it.
 - ▶ Because the shared functionality is remote, network latency can slow its performance.
 - ▶ If the shared service is unavailable, microservices that need the shared functions cannot operate.

Code reuse with a shared library

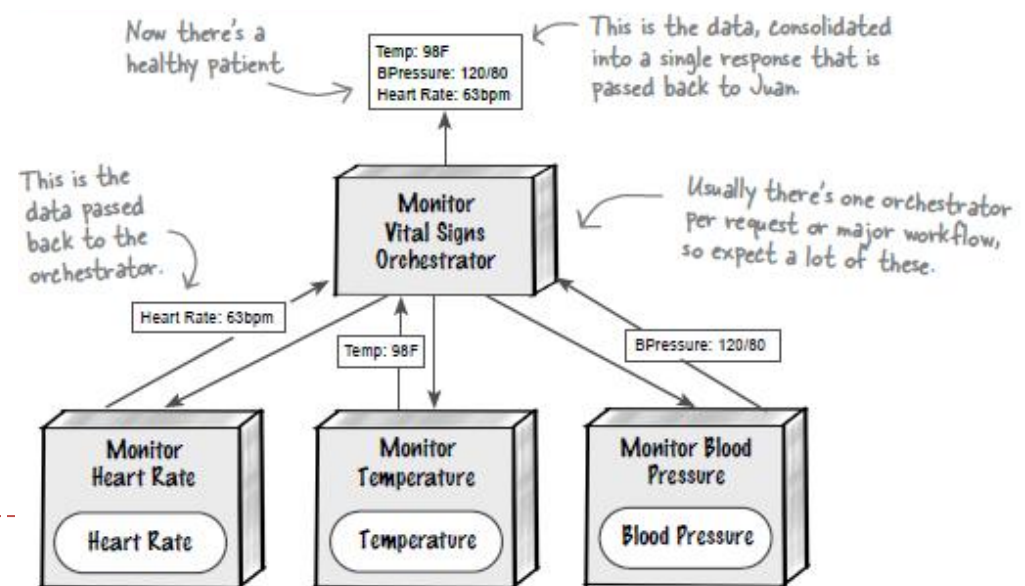
- ▶ A **shared library** is an independent artifact (like a JAR file in Java or a DLL in C#) that is included with each microservice at compile time.
 - ▶ Once the microservice is deployed, *each* microservice has *all* of the shared functionality available to it.
- ▶ Pros
 - ▶ Performance, availability, and scalability are better because the shared functionality is not remote.
 - ▶ Changing code in a shared library is less risky, because shared libraries can be versioned to provide agility and backward compatibility.
- ▶ Cons
 - ▶ You'll need multiple shared libraries if your microservices are written in different programming languages or use different platforms.
 - ▶ Managing dependencies between microservices and shared libraries can become difficult if you have a lot of microservices.
 - ▶ If you change a shared functionality, you must retest and redeploy the microservices that use it.

Managing workflows

- ▶ A ***workflow*** is when fulfilling a single business request needs more than one microservice.
- ▶ You can manage a workflow by using ***orchestration*** or ***choreography***.

Orchestration

- ▶ **Orchestration** is about coordinating all the microservices needed for a workflow.
- ▶ A centralized microservice—the **orchestrator**— does this.
 - ▶ An **orchestration service** is a separate microservice that is responsible for calling all microservices involved in the workflow.
 - ▶ It handles errors and passes data back to the caller (e.g., the user interface).



Orchestration

► *Pros*

- **Centralized workflow.** Request workflows are centralized and understood.
- **Error handling.** Error handling is consolidated into the orchestrator.
- **Workflow state.** The orchestrator knows where the request is in the workflow. If there's a failure, restarting the request where the workflow left off is easy.
- **Workflow changes.** It's easy to change: all changes occur in one central place.

► *Cons*

- **Performance.** Orchestration tends to be slowed by communication between the orchestrator and the microservices, and because the orchestrator typically saves the workflow state to a database each time something changes.
- **Tight coupling.** Because the orchestrator and microservices need to communicate constantly, orchestration tends to be highly coupled.
- **Scalability.** The orchestrator can become a bottleneck as requests increase.
- **Availability.** If the orchestration microservice is unavailable, the request cannot be processed.

Choreography

- ▶ The microservices communicate with each other.
 - ▶ Each performs its function, then calls the next microservice to perform *its* function, until the workflow is complete.

Choreography

► *Pros*

- **Responsiveness.** There's no central orchestrator to communicate with continually: responsiveness and performance tend to be better.
- **Scalability.** Each microservice can scale to meet its throughput demands, independent of other microservices in the workflow.
- **Loose coupling.** Since microservices don't depend on a central orchestration service to direct them, the system tends to be less coupled.

► *Cons*

- **Error handling.** Each microservice is responsible for managing the error workflow if an error occurs. This can lead to too much communication between services.
- **State management.** It's hard to know what state the workflow is in when using choreography, because there's no central conductor controlling the workflow.
 - Usually, one of the microservices (typically the first one in the call chain) is designated the ***state owner***, and other microservices send it their state.
- **Recoverability** When a user retries a request that has failed or is still in progress, it's really hard for the system to know where to restart.

Microservices pros

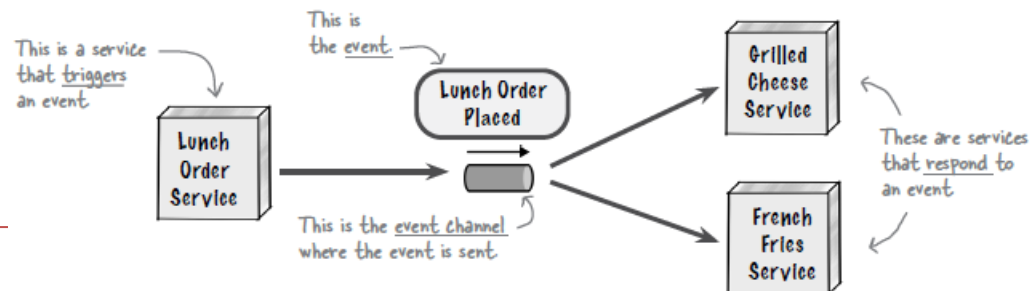
- ▶ **Maintainability.** Microservices are single-purpose and separately deployed: you can easily locate code that needs to change for a particular function.
- ▶ **Testability.** A microservice's testing scope is much smaller than that of a larger monolithic application or a system with large services.
- ▶ **Deployability.** Because microservices are deployed as separate units of software, there are fewer risks involved with releasing a microservice than with large monolithic systems.
- ▶ **Evolvability.** To add functionality to a microservices architecture you create a new service, test it, and deploy it alongside other existing microservices in your system.
- ▶ **Fault tolerance.** If a particular microservice fails, it doesn't bring down the entire system—only that function. Users can continue to use other functions.
- ▶ **Scalability.** Microservices scale at a *function* level rather than a *system* level. This means that you scale only the functionalities you need to meet increased user load and demand, saving resources and lowering costs.

Microservices cons

- ▶ **Complexity.** Microservices involve many hard decisions—about granularity, transactions, workflow, contracts, shared code, communication protocols, team topologies, and deployment strategies...
- ▶ **Monolithic databases.** Each microservice must own its own data and form a physical bounded context. If your *data* can't be broken apart for whatever reason, stay away from this architectural style.
- ▶ **Performance.** The more microservices communicate with each other, the worse their performance will be. They may have to wait for the network, undergo additional security checks, and make extra database calls.
- ▶ **Technically partitioned teams.** Microservices architecture requires cross-functional teams. Each team owns its own group of microservices, all the way from the user interface to the database.
- ▶ **Complex workflows** *Workflows* occur when you need to call multiple microservices for a single business request. If the functionality of your system is too tightly coupled, breaking it into separately deployed services will only result in a big ball of distributed mud.

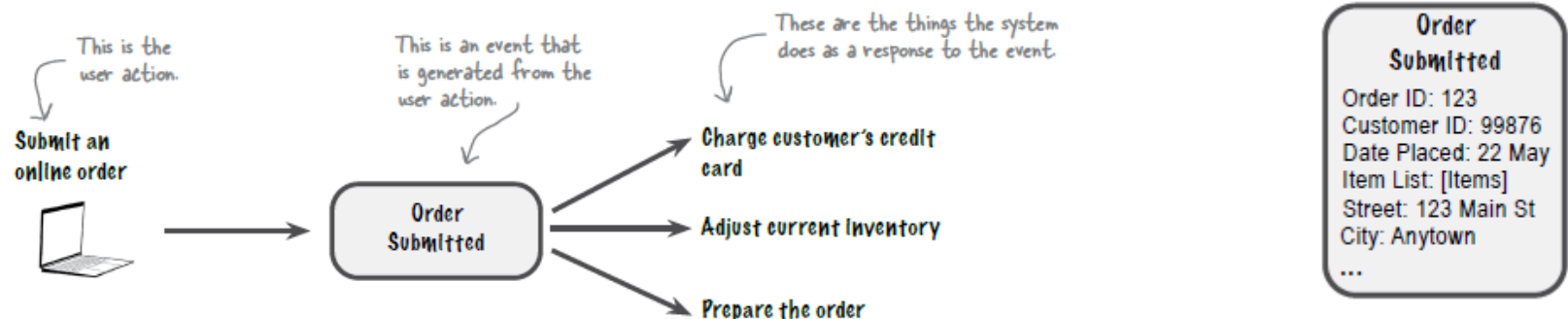
Event-driven Architecture (EDA)

- ▶ *Event-driven architecture* is a popular distributed architectural style.
 - ▶ It's very fast, highly scalable, and easy to extend—but it's also quite complex.
- ▶ **Event-driven architecture (EDA)** break up processing into separate *services*, with each of those services performing its function at the same time by responding to an **event** (something that just happened).
- ▶ Services communicate *asynchronously* through an **event channel**, meaning they don't wait for responses from other services to complete their work.



Events

- ▶ An *event* is something that happens, especially something of importance.
 - ▶ In software systems, certain user actions trigger events—things that happen, like placing a bid for an item up for auction, filing an insurance claim, or making a purchase.
- ▶ Events are a way for a service to let the rest of the system know that something important has just happened.
 - ▶ Events are the means of passing information to other services.
 - ▶ An event usually contains data, like *all* the details of an online order.

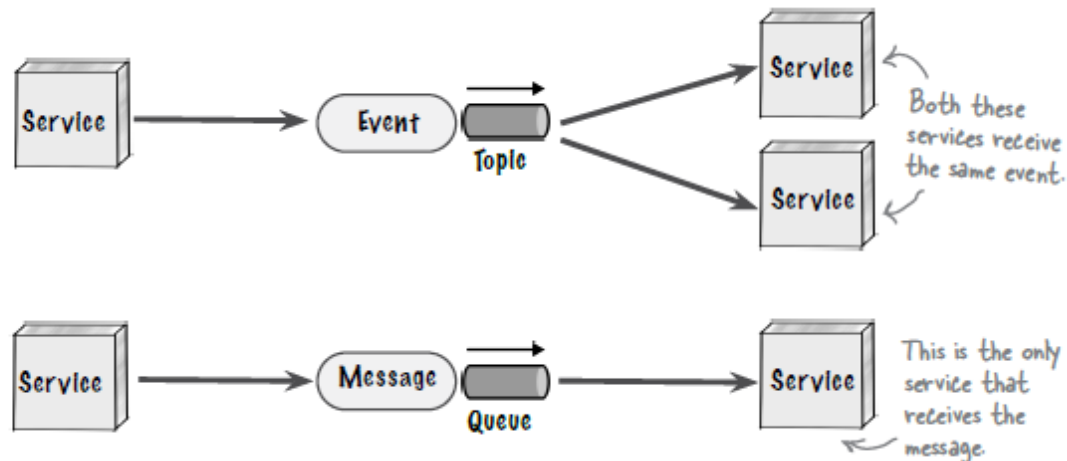


Events

- ▶ An event is not the same thing as a message.
 - ▶ An **event** is used to **broadcast** some action that a service just performed to other services in the system.
 - ▶ For example, a service might tell the rest of the system: *A customer just placed an order.*
 - ▶ A service sending an event **never waits** for a response.
 - ▶ The service generally has no knowledge about what other services (if any) are listening for that event, or what they'll do with that information if they respond to it.
- ▶ A **message**, on the other hand, is a **command**, such as *Apply the payment for this order*, or a **request**, like *Give me shipping options for this customer*.
 - ▶ Because messages are only meant to reach one other service, the other services in the system are unaware of the message.
 - ▶ Services sometimes stop and wait for a response (for instance, if they are sending a request for information). Other times, the service might just issue a command and trust that the receiving service will do its job.

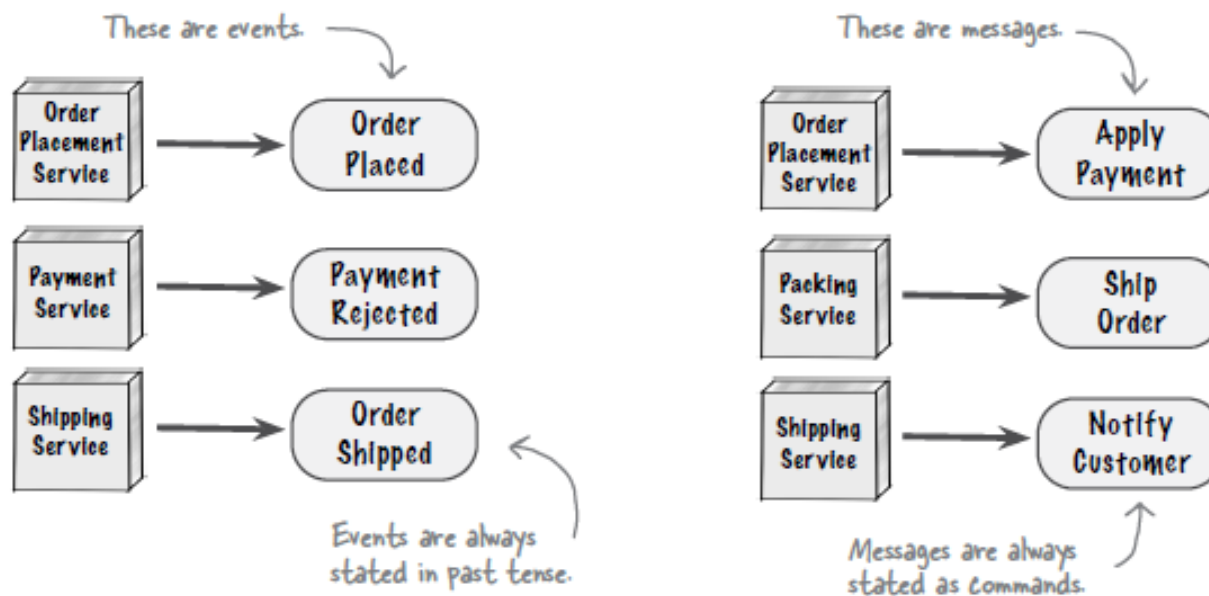
Events

- 1. Events are broadcast to other services using topics, whereas messages are sent to a single service using queues.



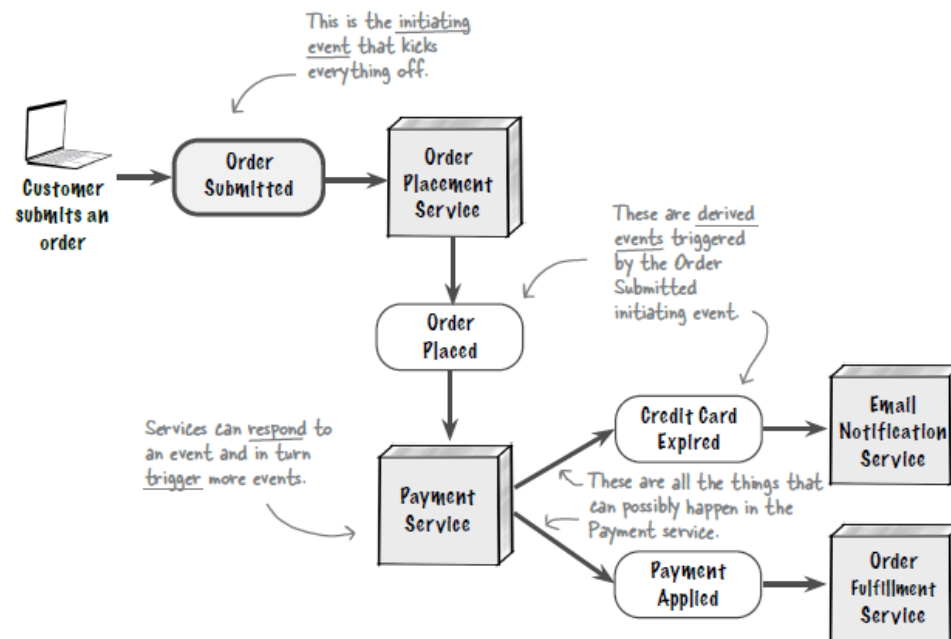
Events

- Events always broadcast something that has already happened, whereas messages request something that needs to be done.



Initiating and derived events

- ▶ Events that originate from an end user are called *initiating events*.
 - ▶ These events kick off a business process.
- ▶ Once a service responds to an initiating event, it might in turn broadcast what it did to the rest of the system, within the scope of that initiating event.
 - ▶ These events are called *derived events*



Asynchronous communication

- ▶ Event-driven architecture is fast because it uses mostly asynchronous (or “async” for short) communication.
 - ▶ REST or HTTP, when you need to call an API or another service are forms of *synchronous communication*.
- ▶ With synchronous (or “sync”) communication, when a service sends information, it must wait for a response from the receiving service before doing anything else.
- ▶ This slows systems down and makes them less scalable.
- ▶ ***Asynchronous communication*** is a fancy way of saying that services don’t wait for a response or acknowledgment from other services when sending them information.
 - ▶ This creates systems that are highly decoupled and very fast.

Asynchronous communication

- ▶ Asynchronous communication is one of the foundations of event-driven architecture.
 - ▶ When a service broadcasts information to other services, it doesn't wait for a response, nor does it care whether the services are available.
 - ▶ This is known as *fire-and-forget* communication—the event is sent (that's the *fire* part), and the service moves on to do other things (that's the *forget* part).
- ▶ Synchronous communication means that the sending service must stop and wait for a response from the service it's calling before continuing its work.
 - ▶ This means that the service being called *must* be available to respond, or an error occurs.

Asynchronous communication

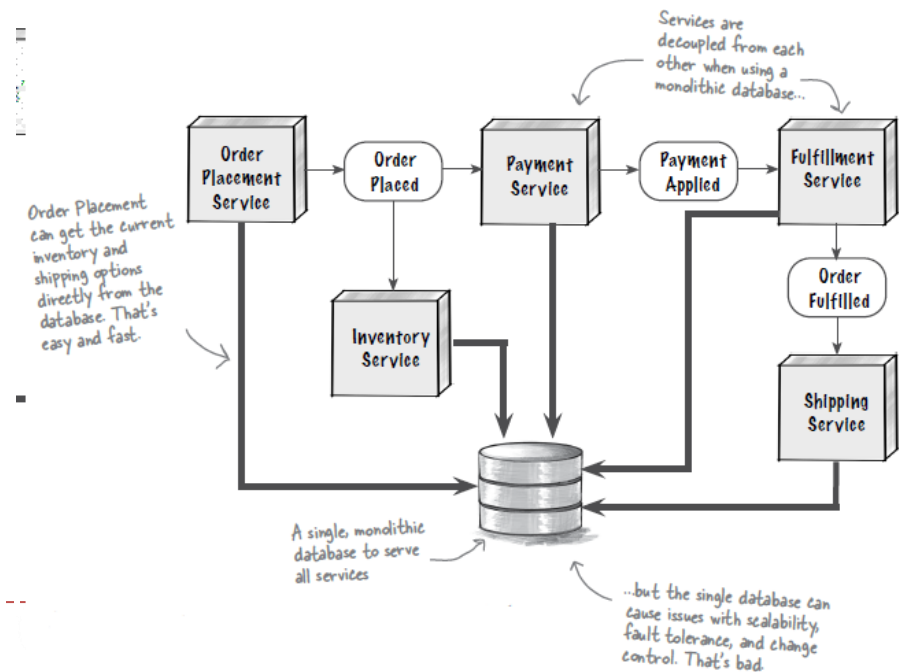
- ▶ Communicating between services asynchronously means better **responsiveness**—in async, it takes less time to complete a request.
- ▶ The main disadvantage of async communication is **error handling**.
 - ▶ With sync communication, if there's a problem, the customer knows right away and has a chance to fix it and resubmit the order.
 - ▶ With async, the customer thinks everything is fine because the system hasn't told them otherwise.
- ▶ This makes error handling much more complex.

Database topologies

- ▶ Data can be a complex topic in EDA.
- ▶ Because EDA is asynchronous, services are highly *decoupled* from *one another*.
- ▶ However, if all the services share a single database, then they end up being highly *coupled* to the *database*.
 - ▶ On the other hand, if each service owns its own data, like with microservices, then services become highly coupled to each other because they need to synchronously ask each other for the data.
 - ▶ In either case, data forms a coupling point—something we try to avoid in EDA. This happens with:
 - ▶ *monolithic databases*,
 - ▶ *domain-partitioned databases*,
 - ▶ *database-per-service* pattern.

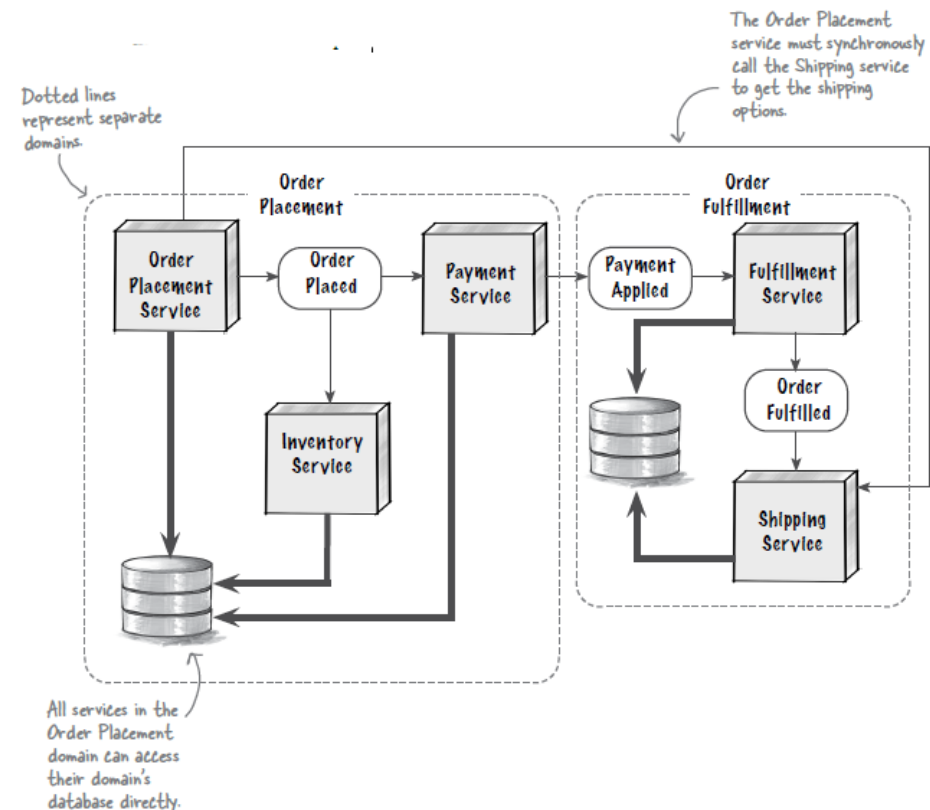
Monolithic database

- ▶ In the *monolithic database* topology, *all services share a single database*.
 - ▶ The main advantage is that when services need data they don't own, they can go directly to the database.
 - ▶ This means they don't have to make synchronous calls to other services to get data.
- ▶ This decoupling comes with a steep price: *managing database changes*.
 - ▶ When you make a change to the structure of one table, it's difficult to identify all the services that change will affect.



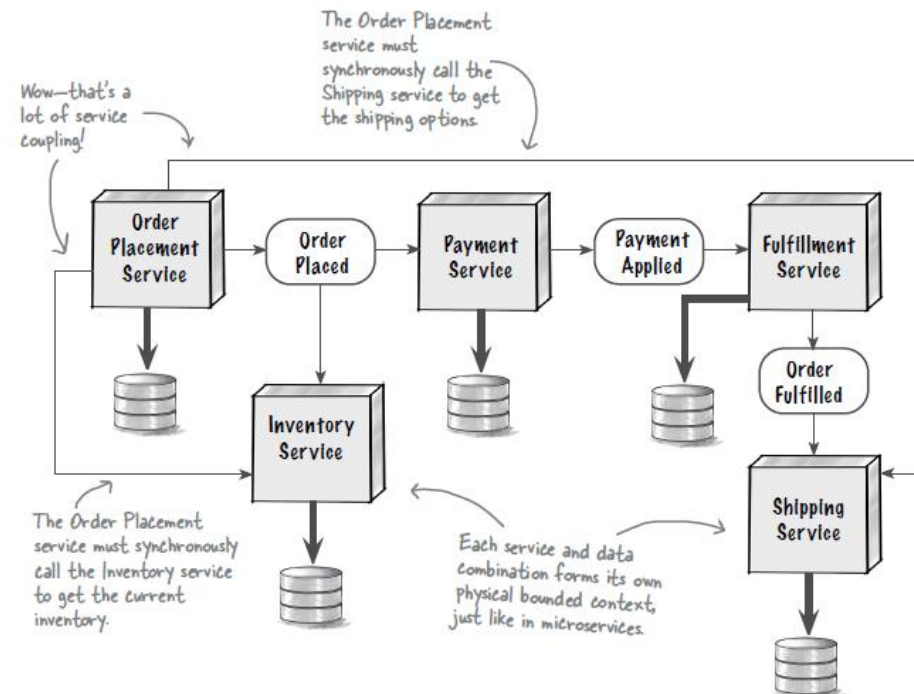
Domain-partitioned databases

- ▶ With *domain-partitioned* databases, each *domain* in the system has its own database.
- ▶ This means any service that belongs to a particular domain will share the database for that domain.
- ▶ However, since each domain forms its own broad physical bounded context, a service in one domain can't directly access a database to get data from another domain.
- ▶ This means it must make a synchronous call to another service to get the data—and now these services are coupled.



Database-per-service

- ▶ Every service has its own database, forming an even tighter physical bounded context than with the domain-partitioned topology.
 - ▶ You get better fault tolerance and better scalability, too.
- ▶ Whenever services need additional data they don't have, they have to *ask* for that data from the service that owns it using synchronous calls.
- ▶ That results in a lot of coupling and communication between services, not to mention much slower performance.



EDA versus microservices

► Performance

- EDA combines asynchronous processing with the ability to do multiple things at once, creating very fast systems.
- Microservices, however, because of their bounded contexts and fine-grained nature, frequently need to communicate *synchronously*.
- This creates a lot of latency, which slows the system down considerably.

► Physical bounded contexts

- In EDA, a physical bounded context is certainly not foundational /required.
- Data sharing is pretty typical in EDA, this architecture doesn't restrict data ownership as strictly as microservices does.

EDA versus microservices

▶ Data granularity

- ▶ A microservices architecture requires each service to own its own data.
- ▶ In EDA, you can choose a single monolithic database, domain-partitioned databases, *or* the database-per-service pattern.

▶ Service granularity

- ▶ A microservice is a *single-purpose service that does one thing really well*.
 - ▶ As a result, microservices tend to be fine-grained.
- ▶ EDA has no such restrictions.
 - ▶ Services in an event-driven architecture (formally called ***event processors***) can be whatever size they need to be—fine-grained, coarse-grained, it doesn't matter.

EDA versus microservices

▶ Event versus request processing

- ▶ Event-driven architecture is built on *event processing*—responding to something that has happened, and in turn triggering more events.
- ▶ Microservices architecture is built on *request processing*—responding to something that needs to happen, like a command or a request, and processing that request.

▶ Communication style

- ▶ EDA uses asynchronous communication between services
- ▶ Microservices typically rely on synchronous communication using REST.
- ▶ EDA can occasionally use synchronous calls for things like retrieving data it doesn't have access to, and microservices can use asynchronous communication when commands don't require a response.
 - ▶ But those are exceptions rather than the rule.

Two topologies

- ▶ The mediator topology is commonly used when you require control over the workflow of an event process
- ▶ The broker topology is used when you require a high degree of responsiveness and dynamic control over the processing of an event.

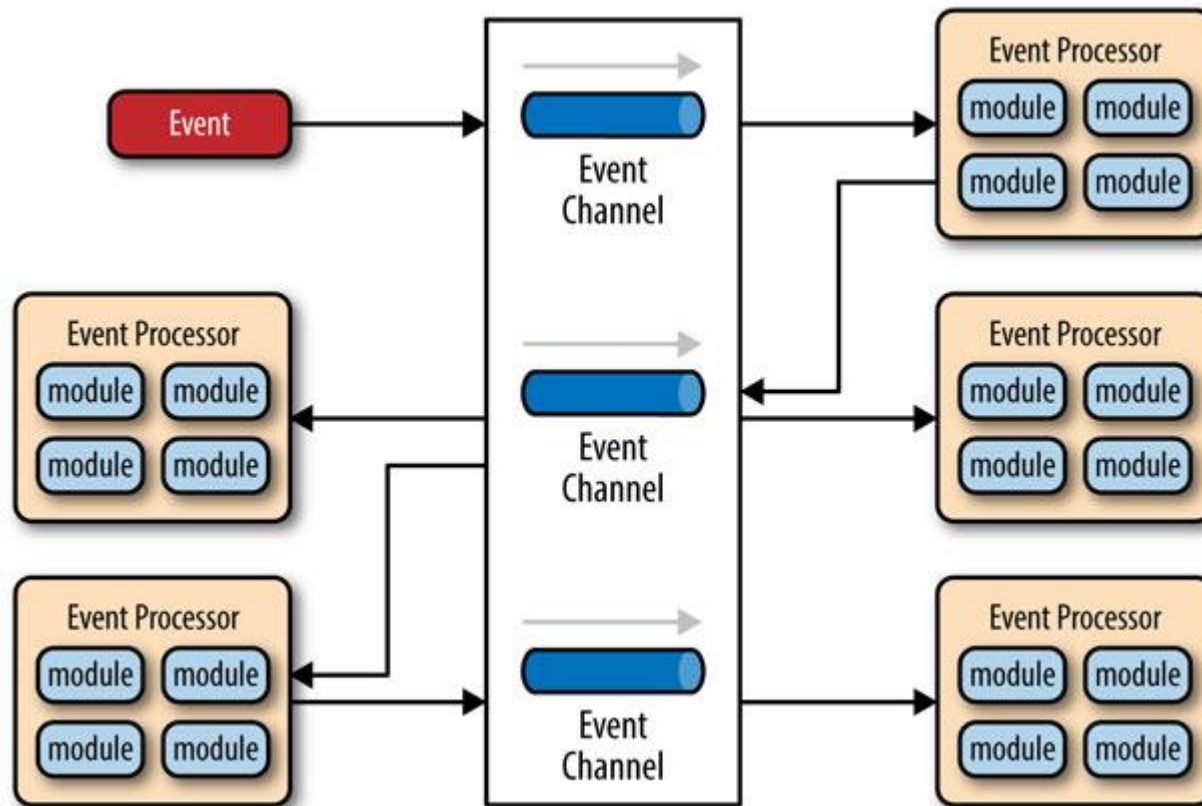
Broker topology

- ▶ The broker topology differs from the mediator topology in that there is no central event mediator.
 - ▶ the message flow is distributed across the event processor components in a chain-like fashion through a lightweight message broker.
 - ▶ This topology is useful when you have a relatively simple event processing flow and you do not want (or need) central event orchestration.
- ▶ There are four main types of architectural components:
 - ▶ an initiating event,
 - ▶ the event broker,
 - ▶ an event processor,
 - ▶ a processing event

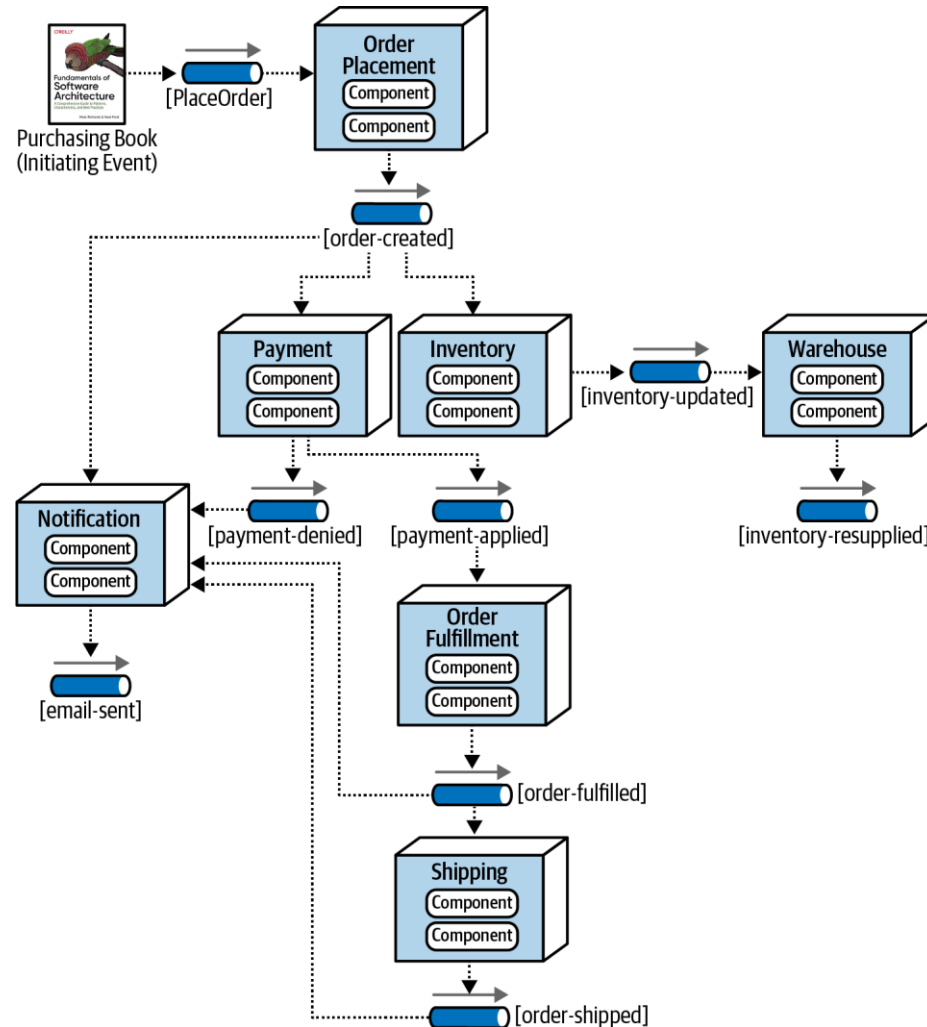
Broker topology

- ▶ The initiating event starts the entire event flow.
- ▶ It is sent to an event channel in the event broker for processing.
 - ▶ There is no mediator component managing and controlling the event.
- ▶ An event processor accepts the initiating event from the event broker and begins the processing of that event.
- ▶ The event processor
 - ▶ performs a specific task associated with the processing of that event,
 - ▶ asynchronously advertises what it did to the rest of the system by creating a processing event.
- ▶ This processing event is asynchronously sent to the event broker for further processing, if needed.
 - ▶ Other event processors listen to the processing event, react to that event by doing something, then advertise through a new processing event what they did.
- ▶ This process continues until no one is interested in what a final event processor did.

Broker topology



Broker topology - example



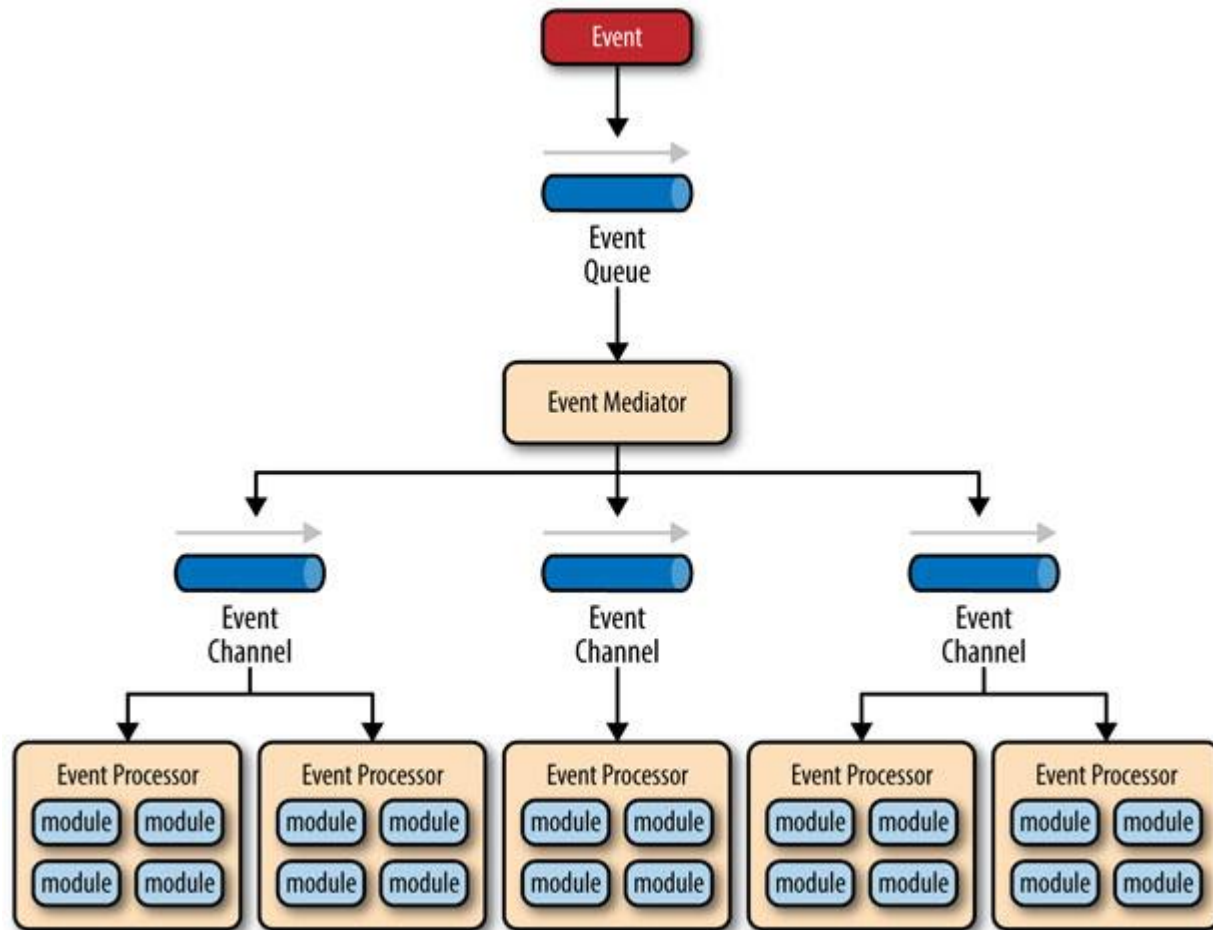
Broker topology

- ▶ Benefits of the broker topology: performance, responsiveness, and scalability.
- ▶ Nevertheless:
 - ▶ There is no control over the overall workflow associated with the initiating event.
 - ▶ It is very dynamic based on various conditions, and no one in the system really knows when the business transaction of placing an order is actually complete.
 - ▶ Error handling is also a big challenge with the broker topology.
 - ▶ If a failure occurs, no one in the system is aware of that crash.
 - ▶ The business process gets stuck and is unable to move without some sort of automated or manual intervention.
 - ▶ The ability to restart a business transaction (recoverability) is not supported.

Mediator topology

- ▶ Central to this topology is an event mediator, which manages and controls the workflow for initiating events that require the coordination of multiple event processors.
- ▶ The architecture components that make up the mediator topology are:
 - ▶ an initiating event,
 - ▶ an event queue,
 - ▶ an event mediator,
 - ▶ event channels,
 - ▶ event processors.
- ▶ The initiating event is sent to an initiating event queue, which is accepted by the event mediator.
- ▶ The event mediator only knows the steps involved in processing the event and therefore generates corresponding processing events that are sent to dedicated event channels (usually queues) in a point-to-point messaging fashion.
- ▶ Event processors then listen to dedicated event channels, process the event, and usually respond back to the mediator that they have completed their work.

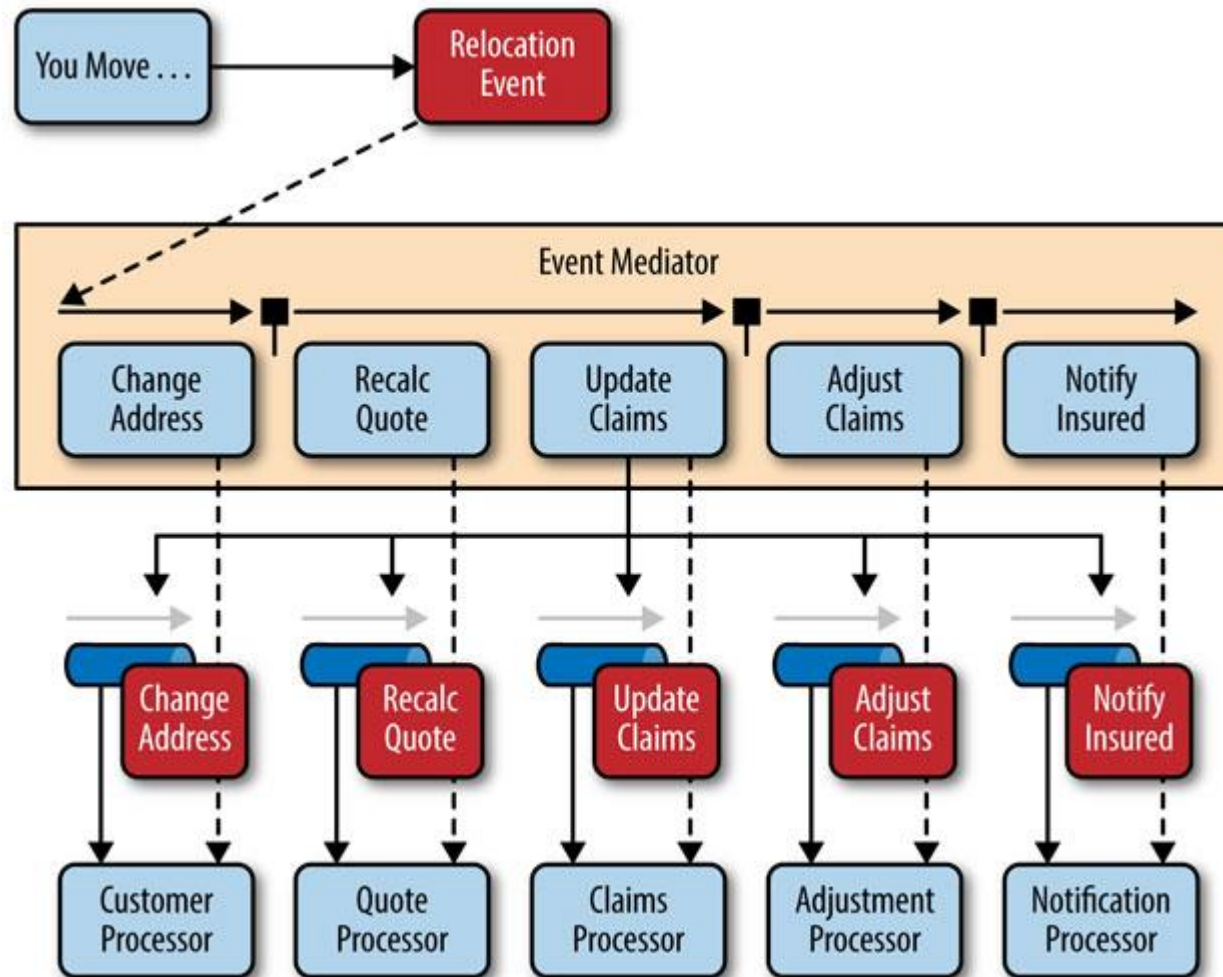
Mediator topology



Mediator topology

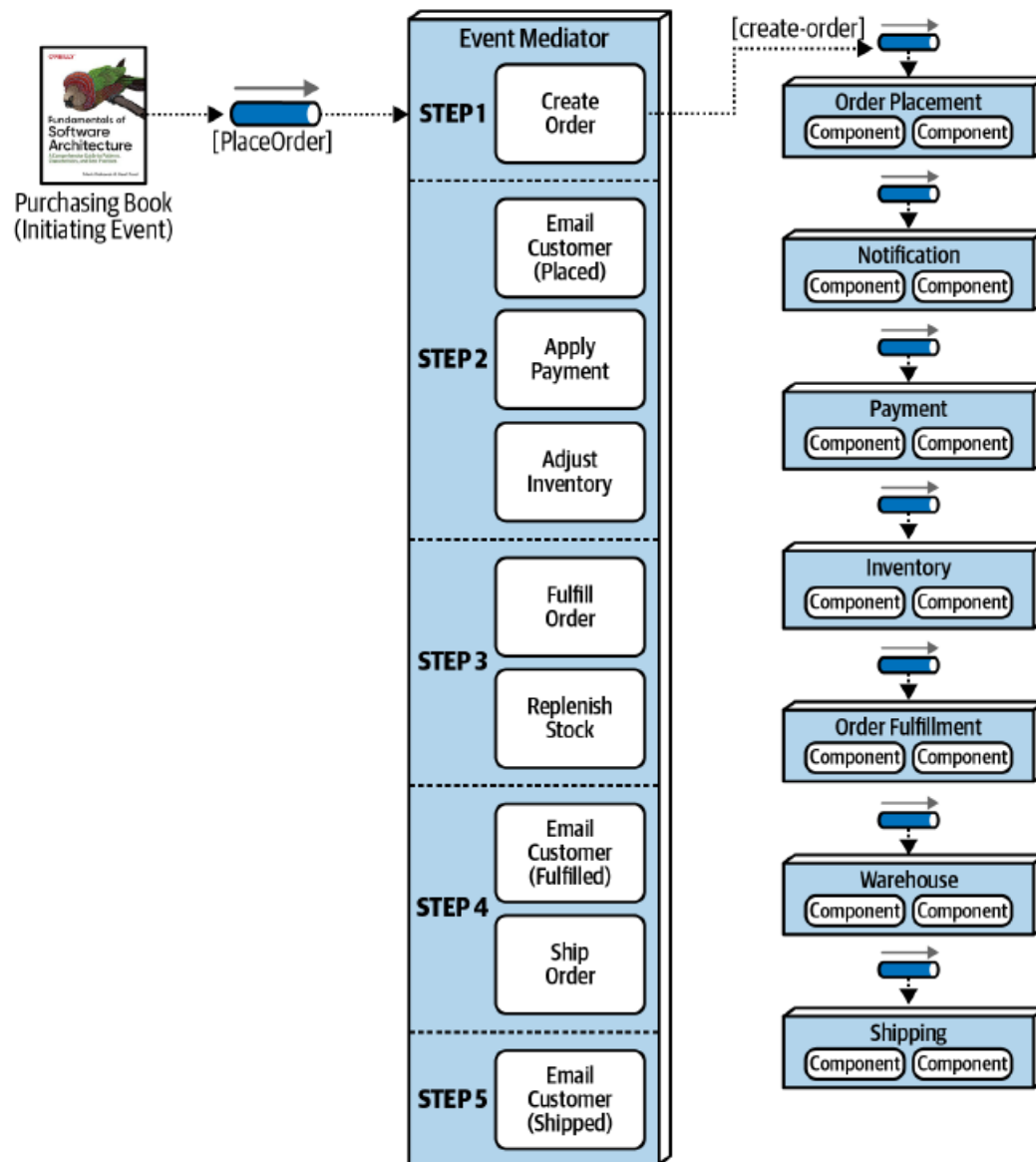
- ▶ In most implementations, there are multiple mediators, usually associated with a particular domain or grouping of events.
 - ▶ This reduces the single point of failure issue associated with this topology
 - ▶ This increases overall throughput and performance.
 - ▶ there might be a customer mediator that handles all customer-related events (such as new customer registration and profile update), and another mediator that handles order-related activities (such as adding an item to a shopping cart and checking out).
- ▶ The event mediator can be implemented in a variety of ways, depending on the nature and complexity of the events it is processing.
 - ▶ Complex mediators are based on Business Process Execution Language (BPEL), an XML-like structure that describes the steps involved in processing an event.

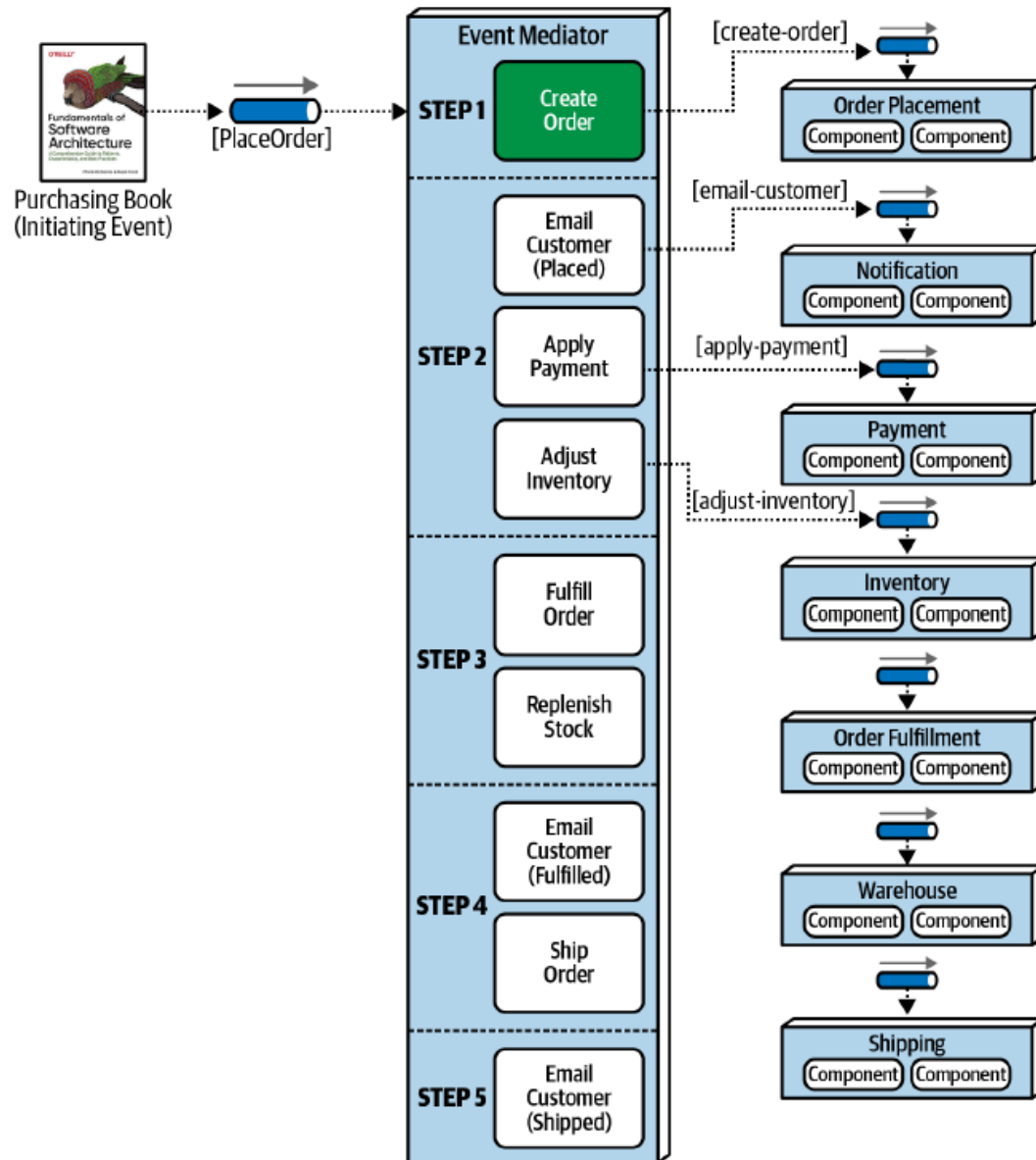
Mediator topology -example

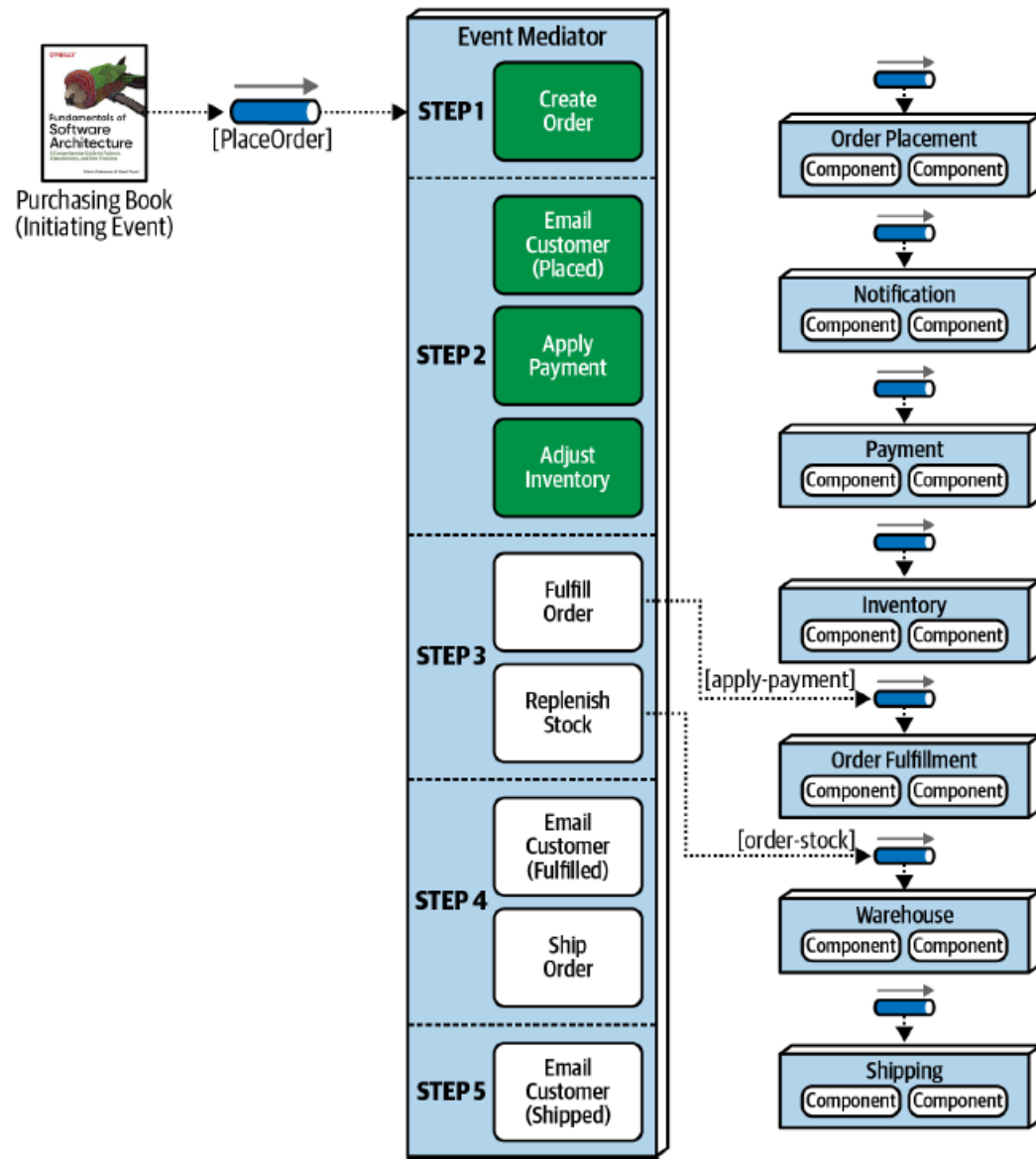


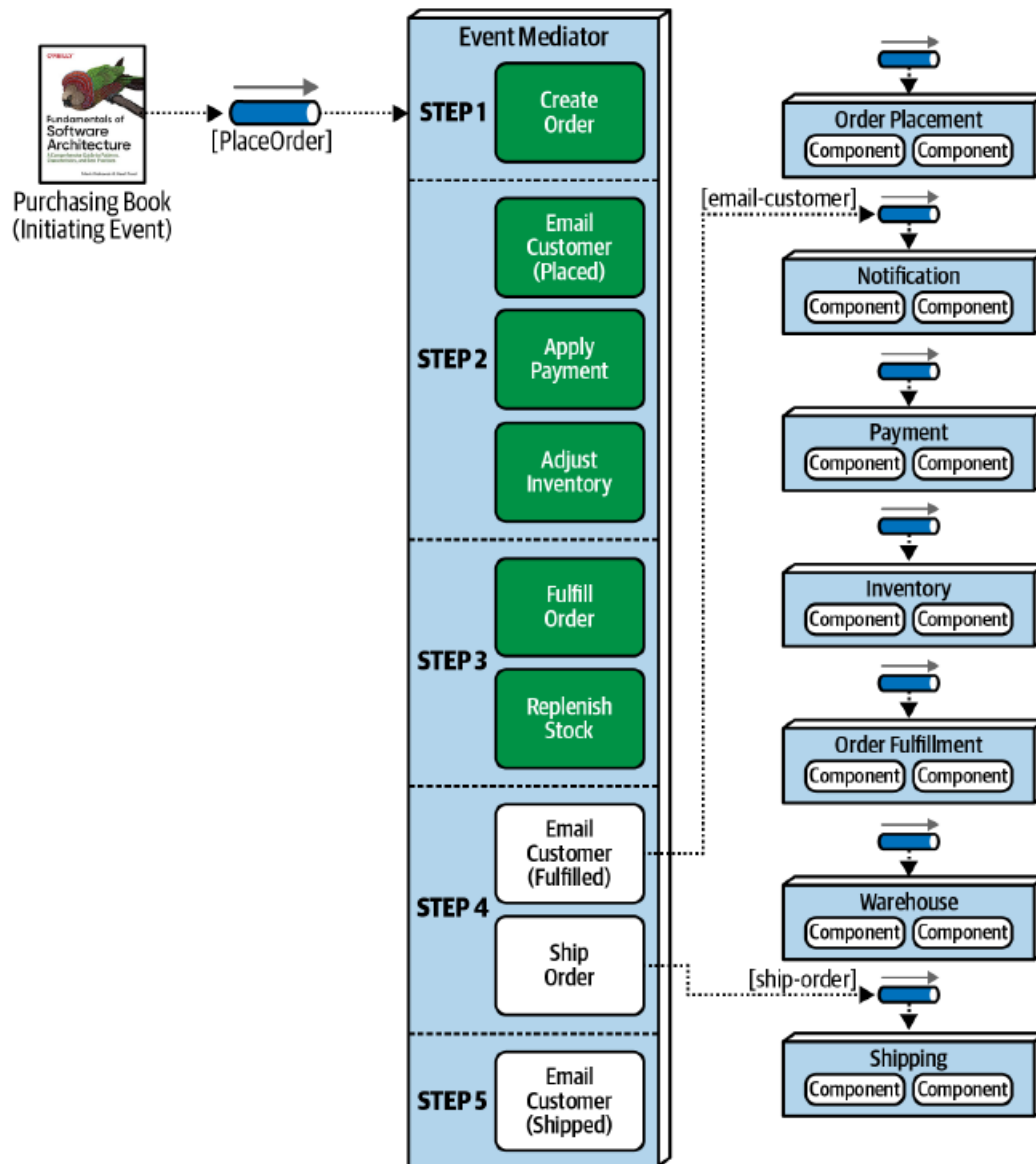
Mediator topology

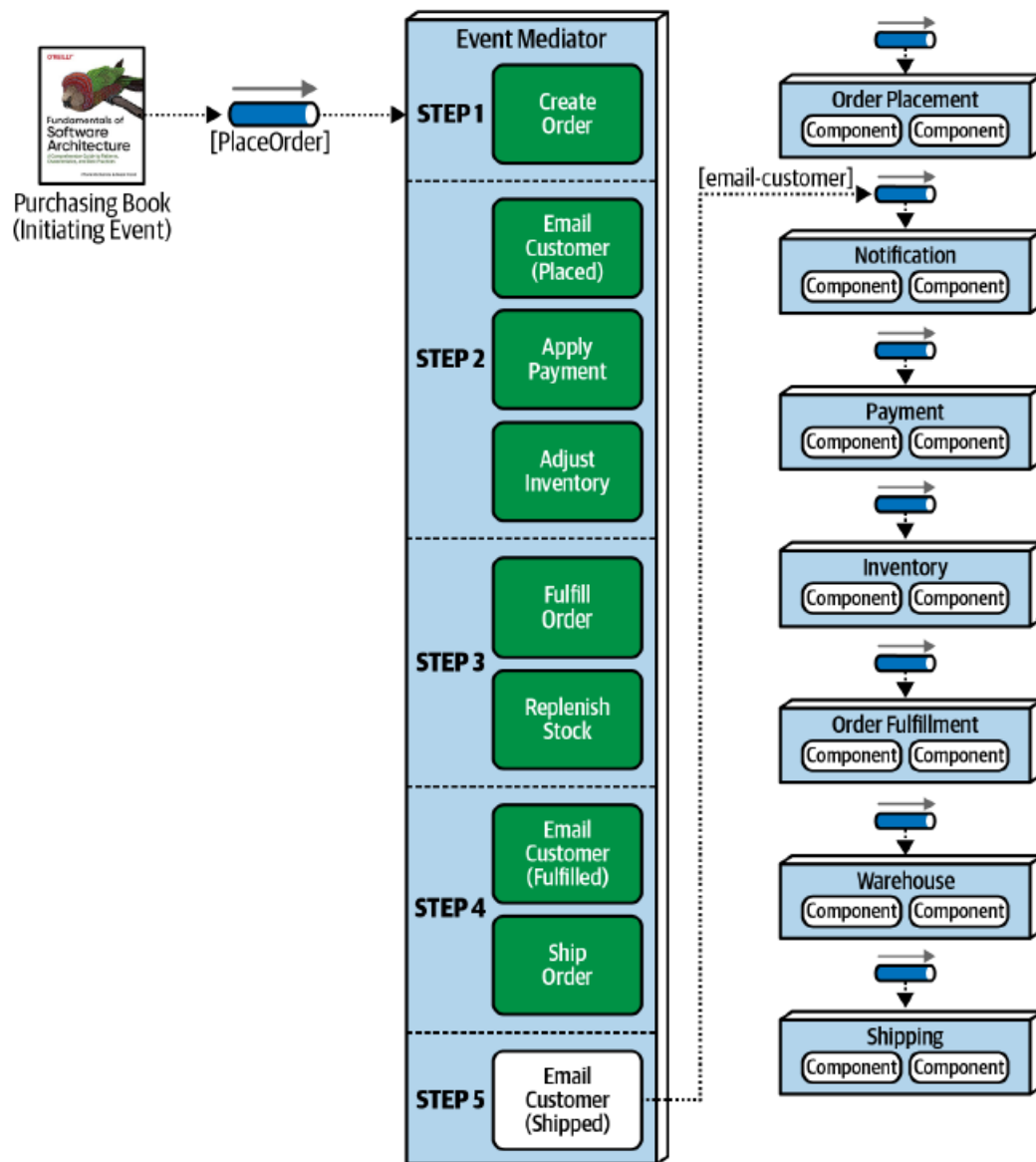
- ▶ The mediator component has knowledge and control over the workflow.
 - ▶ It can maintain event state and manage error handling, recoverability, and restart capabilities.
- ▶ Nevertheless:
 - ▶ It is very difficult to declaratively model the dynamic processing that occurs within a complex event flow.
 - ▶ Although the event processors can easily scale in the same manner as the broker topology, the mediator must scale as well, something that occasionally produces a bottleneck in the overall event processing flow.
 - ▶ Event processors are not as highly decoupled in the mediator topology as with the broker topology, and performance is not as good due to the mediator controlling the processing of the event.











Event-driven architecture pros

- ▶ **Maintainability.** Services in EDA are highly decoupled making them fairly independent and easy to maintain.
- ▶ **Performance.** Because EDA mostly uses asynchronous communication and can multitask, it's very fast.
- ▶ **Evolvability.** EDA services always trigger derived events, onto which we can easily add functionality.
- ▶ **Fault tolerance.** Services are highly decoupled: if one service goes down, it doesn't bring down other services in the workflow.
- ▶ **Scalability.** Event-driven architectures are highly scalable because of asynchronous processing and service decoupling.
 - ▶ Each service can scale independently of others, with the event channels acting as pressure release valves if bottlenecks occur.

Event-driven architecture cons

- ▶ **Databases.** Regardless of the database topology, services are coupled: either to the database or to each other.
- ▶ **Testability.** It's really hard to test asynchronous processing and parallel tasks, making testability a weakness in EDA.
- ▶ **Synchronous calls.** If you have lots of synchronous calls between services and workflows that require synchronously dependent services, EDA is not for you.
- ▶ **Complexity.** EDA is highly complex because it typically uses asynchronous communication and parallel event processing, and because of its varied database topologies and their trade-offs.

Robert C. Martin: Clean Architecture: A Craftsman's Guide to Software Structure and Design, Prentice Hall; 2017

The Clean Architecture

Software Architect

- ▶ Software architects guide the team toward a design maximizing productivity.
 - ▶ The architecture of a software is the shape given to that system by those who build it.
 - ▶ The form of that shape is in the division into components, the arrangement of components, and the ways in which components communicate with each other.
- ▶ The purpose of that shape is to facilitate the **development, deployment, operation, and maintenance** of the software.
- ▶ *The strategy behind that facilitation is to leave as many options open as possible, for as long as possible.*
- ▶ The main goal of software architecture is not to make the system work properly: it is to support the life cycle of the system.
 - ▶ Good architecture makes the system easy to understand, easy to develop, easy to maintain, and easy to deploy.
 - ▶ The ultimate goal is to minimize the lifetime cost of the system and to maximize programmer productivity.

Software Architect

▶ DEVELOPMENT

- ▶ A software that is hard to develop is not likely to have a long and healthy lifetime.

▶ DEPLOYMENT

- ▶ To be effective, a software must be deployable. The higher the cost of deployment, the less useful the system is.

▶ OPERATION

- ▶ The architecture has a limited impact on system operations.
 - ▶ Almost any operational difficulty can be resolved by throwing more hardware without impacting the software architecture.
- ▶ The system architecture makes the operation of readily apparent to the developers.
 - ▶ The system architecture should elevate the use cases, the features, and the required behaviors of the system to first-class entities that are visible landmarks for the developers.

Software Architect

▶ MAINTENANCE

- ▶ The never-ending parade of new features and the trail of defects and corrections consume vast amounts of human resources.
- ▶ The primary cost of maintenance is in spelunking and risk.
 - ▶ **Spelunking is the cost of digging through the software, trying to determine the best place and strategy to add a new feature or to repair a defect.**
 - ▶ While making such changes, the likelihood of creating inadvertent defects is always there, adding to the cost of risk.
- ▶ The architecture should mitigates these costs.
 - ▶ By separating the system into components, and isolating those components through stable interfaces, it is possible to reduce the risk of inadvertent breakage.

Keeping options open

- ▶ Software has two types of value: its behavior and its structure.
 - ▶ The structure makes software soft to quickly and easily change the behavior of machines.
 - ▶ Flexibility depends on the shape of the system, the arrangement of its components, and the way those components are interconnected.
- ▶ The way you keep software soft is to leave as many options (the details that don't matter) open as possible, for as long as possible.
- ▶ All software systems can be decomposed into: **policy and details**.
 - ▶ The **policy** element embodies all the business rules and procedures.
 - ▶ The **details** are the elements necessary to enable humans, other systems, and programmers to communicate with the policy, but that do not impact the behavior of the policy at all.
 - ▶ They include IO, databases, web systems, servers, frameworks, communication protocols, and so forth.
- ▶ The goal is to create a **shape that recognizes policy as the most essential element of the system while making the details irrelevant to that policy**.
- ▶ A good architect maximizes the number of decisions not made.

Keeping options open

- ▶ For example:
 - ▶ It is not necessary to choose a database system in the early days of development.
 - ▶ Indeed, if the architect is careful, the high-level policy will not care if the database is relational, distributed, noSQL, or just plain flat files.
 - ▶ It is not necessary to choose a web server early in development, because the high-level policy should not know that it is being delivered over the web.
 - ▶ If the high-level policy is unaware of HTML, AJAX, JSP, JSF, then you don't need to decide which web system to use until much later in the project.
- ▶ If you can develop the high-level policy without committing to the details, you can delay decisions about those details for a long time.
 - ▶ And the longer you wait to make those decisions, the more information you have with which to make them properly.
- ▶ This also leaves you the option to try different experiments.

Independence

- ▶ A good architecture must support:
 - ▶ The use cases of the system, i.e. the intent of the system.
 - ▶ The intent of the system should be visible at the architectural level.
 - ▶ A shopping cart application with a good architecture will *look* like a shopping cart application.
 - Those elements will be classes or functions or modules that have prominent positions within the architecture, and they will have names that clearly describe their function.
 - ▶ The operation of the system.
 - ▶ If the system must handle 100,000 customers per second, the architecture must support that kind of throughput and response time for each use case that demands it.
 - It could mean arranging the processing elements into an array of little services can be run in parallel on many different servers.
 - It could mean a plethora of little lightweight threads sharing the address space of a single process within a single processor.
 - ▶ As strange as it may seem, this decision is one of the options that a good architect leaves open.

Independence

- ▶ A good architecture must support:
 - ▶ The development of the system.
 - ▶ Conway's law says: *Any organization that designs a system will produce a design whose structure is a copy of the organization's communication structure.*
 - ▶ A system is developed by an organization with many teams and many concerns must have an architecture that facilitates independent actions by those teams, so that the teams do not interfere with each other during development.
 - ▶ This is accomplished by properly partitioning the system into well-isolated, independently developable components.
 - Those components can then be allocated to teams that can work independently of each other.
 - ▶ The deployment of the system.
 - ▶ The goal is “immediate deployment.”
 - A good architecture does not rely on dozens of little configuration scripts and property file tweaks.

Leaving option open

- ▶ A good architecture balances all of these concerns with a component structure that mutually satisfies them all.
- ▶ The reality is that achieving this balance is pretty hard.
 - ▶ Most of the time we don't know
 - ▶ what all the use cases are,
 - ▶ the operational constraints,
 - ▶ the team structure,
 - ▶ the deployment requirements.
 - ▶ Even if we did know them, they will inevitably change as the system moves through its life cycle.
 - ▶ In short, the goals we must meet are indistinct and inconstant.

Decoupling layers

- ▶ The architect wants the system structure supports all use cases, but does not know what all those use cases are.
 - ▶ The architect *does* know the basic intent of the system.
 - ▶ The architect can employ the Single Responsibility Principle and the Common Closure Principle to separate those things that change for different reasons, and to collect those things that change for the same reasons—given the context of the intent of the system.
- ▶ What changes for different reasons?
 - ▶ User interfaces change for reasons that have nothing to do with business rules.
 - ▶ UI will be separated from the business rule portions.
 - ▶ Business rules may be closely tied to the application, or general.
 - ▶ The validation of input fields is a business rule that is closely tied to the application itself.
 - ▶ The calculation of interest on an account is a business rules that is associated with the domain.
 - ▶ These two rules change at different rates, and for different reasons—so they should be separated.
 - ▶ The database, the query language, and even the schema are technical details that have nothing to do with the business rules or the UI.
- ▶ **The system should be divided into decoupled horizontal layers** the UI, application-specific business rules, application-independent business rules, and the database, at least.

Decoupling use cases

- ▶ The use cases change for different reasons.
 - ▶ The use case for adding an order changes at a different rate, and for different reasons, than the use case that deletes an order.
- ▶ Use cases are a natural way to divide the system.
 - ▶ Use cases are vertical slices that cut through the horizontal layers of the system.
 - ▶ Each use case uses some UI, some application-specific business rules, some application-independent business rules, and some database functionality.
- ▶ Thus, as we are dividing the system into horizontal layers, we are also dividing the system into vertical use cases that cut through those layers.
- ▶ You can see the pattern here.
 - ▶ If you decouple the elements of the system that change for different reasons, then you can continue to add new use cases without interfering with old ones.
 - ▶ If you also group the UI and database in support of those use cases, then adding new use cases will be unlikely to affect older ones.

Decoupling mode

- ▶ The decoupling helps with operations.
- ▶ If the use cases are separated, then those that must run at a high throughput are likely already separated from those that must run at a low throughput.
 - ▶ If the UI and the database are separated from the business rules, then they can run in different servers.
 - ▶ Those that require higher bandwidth can be replicated in many servers.
- ▶ To take advantage of the operational benefit, the decoupling must have the appropriate mode.
 - ▶ They must be independent services, which communicate over a network of some kind.
- ▶ Many architects call such components “services” or “micro-services,” depending upon some vague notion of line count.
 - ▶ An architecture based on services is often called a service-oriented architecture.
- ▶ *The decoupling mode is one of those options to leave options open.*

Independent developability and deployability

▶ INDEPENDENT DEVELOP-ABILITY

- ▶ When components are strongly decoupled, the interference between teams is mitigated.
 - ▶ If the use cases are decoupled from one another, then a team that focuses on the addOrder use case is not likely to interfere with a team that focuses on the deleteOrder use case.

▶ INDEPENDENT DEPLOYABILITY

- ▶ If the decoupling is done well, then it should be possible to hot-swap layers and use cases in running systems.
- ▶ Adding a new use case could be as simple as adding a few new jar files or services to the system while leaving the rest alone.

Duplication

- ▶ Duplication is generally a bad thing in software.
 - ▶ True duplication: every change to one instance necessitates the same change to every duplicate.
 - ▶ False or accidental duplication: i two duplicated codes evolve along different paths—if they change at different rates, and for different reasons.
- ▶ When you are vertically separating use cases, your temptation will be to couple the use cases because they have similar structures, or similar algorithms, or similar database queries and/or schemas.
 - ▶ Make sure the duplication is real.
- ▶ When you are separating layers horizontally, you might notice that the data structure of a particular database record is very similar to the data structure of a particular screen view.
 - ▶ You may be tempted to simply pass the database record up to the UI, rather than to create a view model that looks the same and copy the elements across.
 - ▶ Creating the separate view model will help you keep the layers properly decoupled.

Decoupling Modes

- ▶ **Source level.** We can control the dependencies between source code modules so that changes to one module do not force changes or recompilation of others.
 - ▶ The components all execute in the same address space, and communicate with each other using simple function calls.
 - ▶ There is a single executable loaded into computer memory. People often call this a monolithic structure.
- ▶ **Deployment level.** We can control the dependencies between deployable units such as jar files, DLLs, or shared libraries, so that changes to the source code in one module do not force others to be rebuilt and redeployed.
 - ▶ Many of the components may still live in the same address space, and communicate through function calls.
 - ▶ Other components may live in other processes in the same processor, and communicate through interprocess communications, sockets, or shared memory.
 - ▶ The important thing here is that the decoupled components are partitioned into independently deployable units such as jar files, Gem files, or DLLs.
- ▶ **Service level.** We reduce the dependencies down to the level of data structures, and communicate solely through network packets such that every execution unit is entirely independent of source and binary changes to others (e.g., services or micro-services).

Decoupling Modes

What is the best mode to use?

- ▶ It's hard to know which mode is best during the early phases of a project.
 - ▶ As the project matures, the optimal mode may change.
- ▶ A system that runs on one server right now might grow to the point where some of its components ought to run on separate servers.
 - ▶ It might require decoupling into deployable units, or even services.
- ▶ One solution is to simply decouple at the service level by default.
 - ▶ Service-level decoupling is expensive, both in development time and in system resources. Dealing with service boundaries where none are needed is a waste of effort, memory, ...
- ▶ **A good architecture will allow a system to be born as a monolith, deployed in a single file, but then to grow into a set of independently deployable units, and then all the way to independent services and/or micro-services.**
 - ▶ Later, as things change, it should allow for reversing that progression and sliding all the way back down into a monolith.
- ▶ A good architecture leaves the decoupling mode open as an option so that large deployments can use one mode, whereas small deployments can use another.

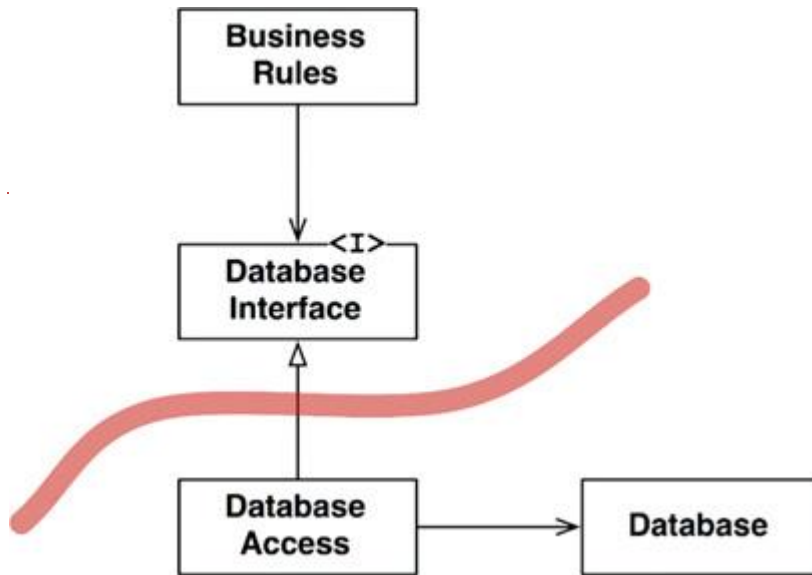
Boundaries: drawing lines

- ▶ Software architecture is the art of drawing *boundaries*.
 - ▶ They separate software elements from one another, and restrict those on one side from knowing about those on the other.
- ▶ The goal of an architecture is to minimize the human resources required to build and maintain the required system.
 - ▶ *Coupling*—and especially to premature decisions saps this goal.
- ▶ Which kinds of decisions are premature?
 - ▶ Decisions that have nothing to do with the business requirements—the use cases—of the system.
 - ▶ These include decisions about frameworks, databases, web servers, utility libraries, dependency injection, and the like.
- ▶ A good system architecture allows those decisions to be made at the latest possible moment, without significant impact.

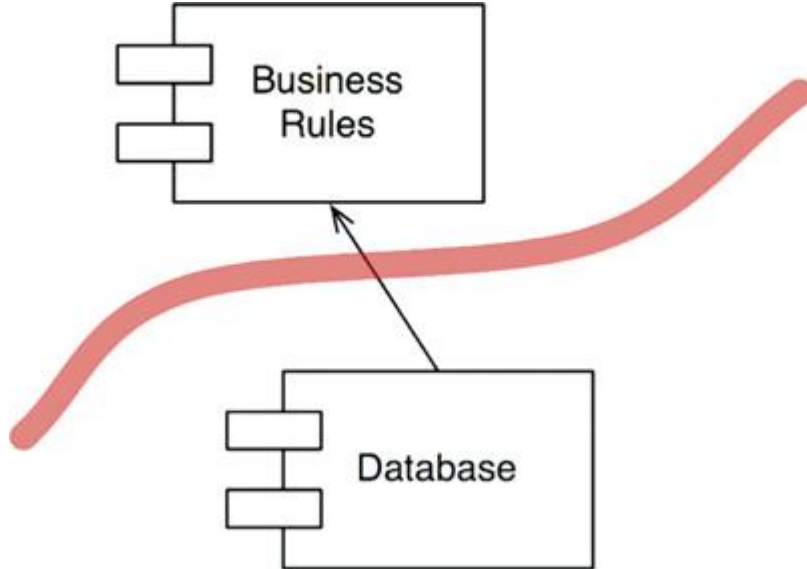
Boundaries: drawing lines

- ▶ You draw lines between things that matter and things that don't.
 - ▶ The GUI doesn't matter to the business rules, so there should be a line between them.
 - ▶ The database doesn't matter to the GUI, so there should be a line between them.
 - ▶ The database doesn't matter to the business rules, so there should be a line between them.

Boundaries: drawing lines

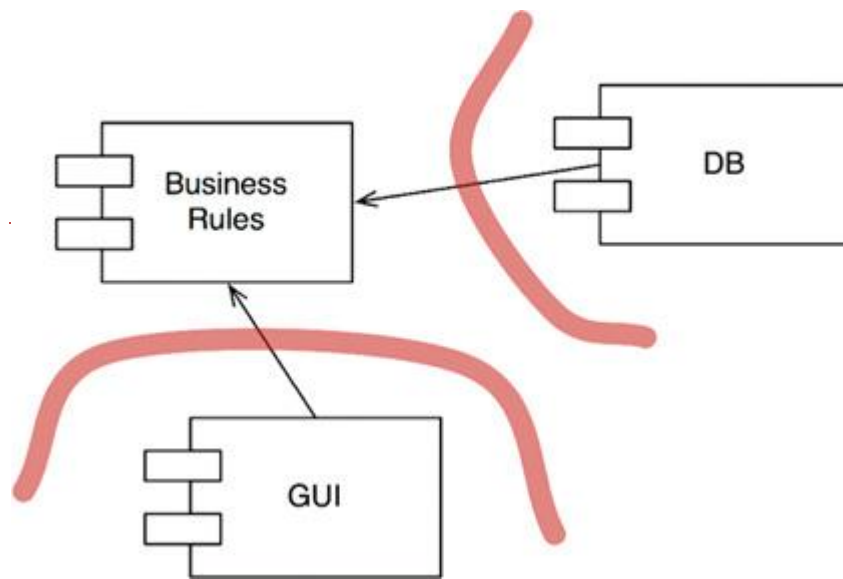


- ▶ The database is a tool that the business rules can use *indirectly*.
 - ▶ The business rules don't need to know about the schema, the query language, or any other detail about the database.
- ▶ The business rules need to know that there is a set of functions that can be used to fetch or save data.
- ▶ This allows us to put the database behind an interface.
 - ▶ Note the two arrows leaving the DatabaseAccess class.
 - ▶ Those two arrows point away from the DatabaseAccess class. That means that none of these classes knows that the DatabaseAccess class exists.



Boundaries: drawing lines

- ▶ At the component level,
 - ▶ The Database knows about the BusinessRules.
 - ▶ The Database does not matter to the BusinessRules, but the Database cannot exist without the BusinessRules
 - ▶ The BusinessRules do not know about the Database.
 - ▶ This implies that
 - ▶ the DatabaseInterface classes live in the BusinessRules component,
 - ▶ the DatabaseAccess classes live in the Database component.
- ▶ With this architecture the BusinessRules could use any kind of database.
 - ▶ The Database component could be replaced with many different implementations—the BusinessRules don't care.



Plugin Architecture

- ▶ The history of software development technology is the story of how to conveniently create plugins to establish a scalable and maintainable system architecture.
 - ▶ The core business rules are kept separate from, and independent of, those components that are either optional or that can be implemented in many different forms
 - ▶ Because the user interface in this design is considered to be a plugin, we have made it possible to plug in many different kinds of user interfaces.
 - ▶ They could be web based, client/server based, SOA based, Console based, or based on any other kind of user interface technology.
 - ▶ The same is true of the database.

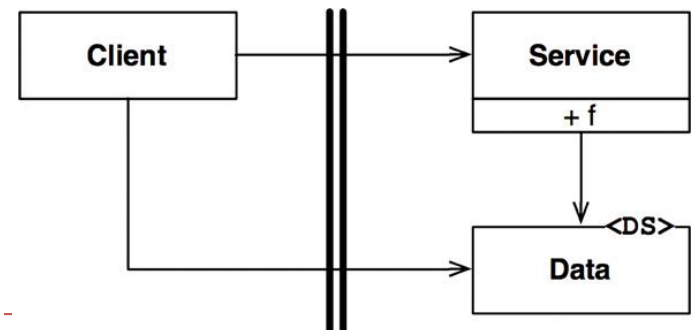
Boundary Anatomy

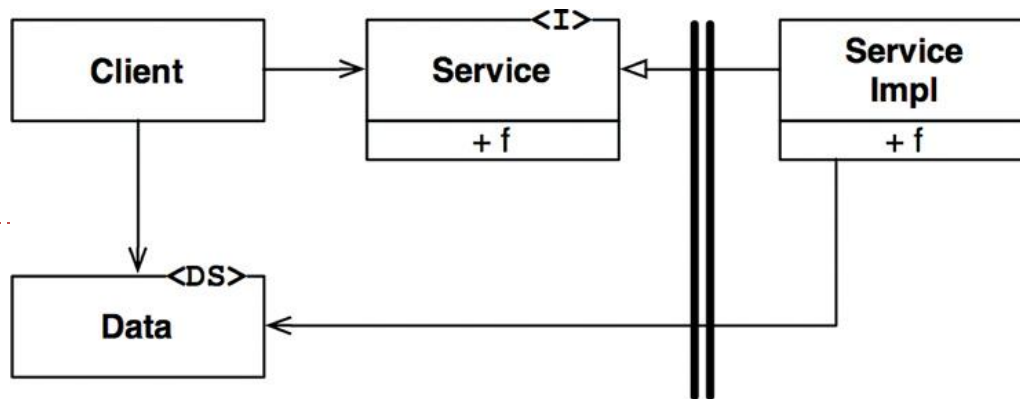
▶ Boundary Crossing

- ▶ At runtime, a boundary crossing is nothing more than a function on one side of the boundary calling a function on the other side and passing along some data.
 - ▶ One source code module changes, other source code modules may have to be changed or recompiled, and then redeployed.
 - ▶ Managing and building firewalls against this change is what boundaries are all about.

Monolith

- ▶ Monolith: The simplest and most common of the architectural boundaries
 - ▶ a disciplined segregation of functions and data within a single processor and a single address space.
- ▶ The simplest possible boundary crossing is a function call from a low-level client to a higher-level service.
 - ▶ Both the runtime dependency and the compile-time dependency point in the same direction, toward the higher-level component.
- ▶ The flow of control crosses the boundary from left to right.
 - ▶ The Client calls function $f()$ on the Service. It passes along an instance of Data.
 - ▶ The $\langle DS \rangle$ marker simply indicates a data structure.
 - ▶ The Data may be passed as a function argument or by some other more elaborated means.





Monolith

- ▶ When a client needs to invoke a service, dynamic polymorphism is used to invert the dependency against the flow of control.
- ▶ The flow of control crosses the boundary from left to right.
 - ▶ The Client calls the `f()` function of the `ServiceImpl` through the `Service` interface.
- ▶ Even in a monolithic, statically linked executable, this kind of disciplined partitioning can greatly aid the job of developing, testing, and deploying the project.
 - ▶ Teams can work independently of each other on their own components.
 - ▶ High-level components remain independent of lower-level details.

Deployment components

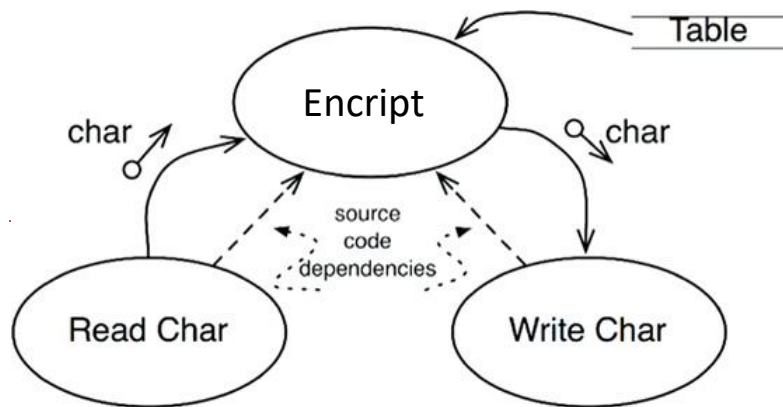
- ▶ The simplest physical representation of an architectural boundary is a dynamically linked library like a .Net DLL, a Java jar file, or a UNIX shared library.
- ▶ Deployment does not involve compilation.
 - ▶ The components are delivered in binary, or other deployable form.
 - ▶ This is the deployment-level decoupling mode.
- ▶ The act of deployment is simply the gathering of these deployable units together in some convenient form, such as a WAR file.
 - ▶ Deployment-level components are the same as monoliths.
 - ▶ The functions generally all exist in the same processor and address space.
 - ▶ The strategies for segregating components and managing dependencies are the same.
- ▶ As with monoliths, communications across deployment component boundaries are just function calls.

Services

- ▶ A service is a process, generally started from the command line or through an equivalent system call.
 - ▶ Services do not depend on their physical location.
 - ▶ Two communicating services may, or may not, operate in the same physical processor or multicore.
 - ▶ The services assume that all communications take place over the network.
- ▶ Communications across service boundaries are very slow compared to function calls.
 - ▶ Turnaround times can range from tens of milliseconds to seconds.
 - ▶ Communications at this level must deal with high levels of latency.

Policy and Level

- ▶ A computer program is a description of the policy by which inputs are transformed into outputs.
- ▶ In most systems, that policy can be broken down into smaller statements of policy that
 - ▶ describe how particular business rules are to be calculated
 - ▶ how certain reports are to be formatted
 - ▶ how input data are to be validated.
- ▶ The architecture design has to separate and group policies based on the ways they change.
 - ▶ Policies that change for the same reasons, and at the same times, are at the same level and belong together in the same component.
- ▶ These regrouped components form a directed acyclic graph.
 - ▶ The nodes of the graph are the components that contain policies at the same level.
 - ▶ The directed edges are the dependencies between those components.
 - ▶ They connect components that are at different levels.
- ▶ Those dependencies are source code, compile-time dependencies
- ▶ In a good architecture, the low-level components depend on high-level components.



Level

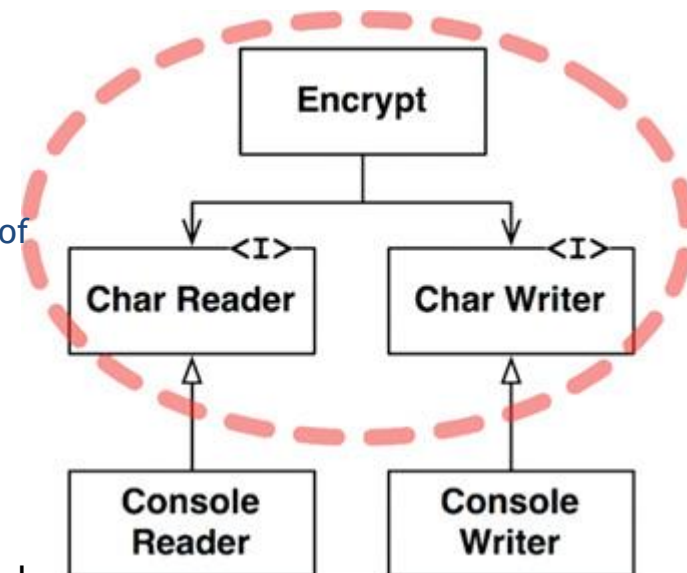
- ▶ A strict definition of “level” is “the distance from the inputs and outputs.”
 - ▶ The farther a policy is from both the inputs and the outputs of the system, the higher its level.
 - ▶ The policies that manage input and output are the lowest-level policies in the system.
- ▶ The data flow diagram depicts a simple encryption program that reads characters from an input device, translates the characters using a table, and writes the translated characters to an output device.
 - ▶ The data flows are shown as curved solid arrows.
 - ▶ The properly designed source code dependencies are shown as straight dashed lines.
- ▶ The Encrypt component is the highest-level component because it is the component that is farthest from the inputs and outputs.
 - ▶ Note that the data flows and the source code dependencies do not always point in the same direction. This, again, is part of the art of software architecture. We want source code dependencies to be decoupled from data flow and coupled to level.

```
function encrypt() {  
    while(true)  
        writeChar(translate(readChar()));  
}
```

Level

► The above is incorrect architecture:

- the high-level encrypt function depends on the lower-level readChar and writeChar functions.
- A better architecture is shown in the class diagram that decouples the high-level encryption policy from the lower-level input/output policies.
 - This makes the encryption policy usable in a wide range of contexts.
 - Higher-level policies tend to change less frequently, and for more important reasons, than lower-level policies.
 - Lower-level policies tend to change frequently, and with more urgency, but for less important reasons.
- Keeping these policies separate, with all source code dependencies pointing in the direction of the higher-level policies, reduces the impact of change.
 - Trivial but urgent changes at the lowest levels of the system have little or no impact on the higher, more important levels.



Business Rules

- ▶ By dividing an application into business rules and plugins, we see what business rules actually are.
 - ▶ Business rules are rules or procedures that make or save the business money.
 - ▶ These rules exists irrespectively of whether they were implemented on a computer.
 - The fact that a bank charges $N\%$ interest for a loan is a business rule that makes the bank money.
 - It doesn't matter if a computer program calculates the interest.
- ▶ *Critical Business Rules* are critical to the business itself, and would exist even if there were no computer system
- ▶ Critical Business Rules can require *Critical Business Data* to work with.
 - ▶ This is the data that would exist even if the system were not automated.
- ▶ The critical rules and data are inextricably bound: so they are a good candidate for an object.
 - ▶ We'll call this kind of object an *Entity*.

Entities

- ▶ An Entity is an object within our computer system that embodies a small set of critical business rules operating on Critical Business Data.
 - ▶ The Entity object contains the Critical Business Data or has very easy access to that data.
 - ▶ The interface of the Entity consists of the functions that implement the Critical Business Rules that operate on that data.
 - ▶ The Loan entity has three pieces of Critical Business Data, and three related Critical Business Rules at its interface.
- ▶ Creating this kind of class, we are gathering together the software that implements a concept that is critical to the business, and separating it from every other concern in the automated system we are building.
 - ▶ This class stands alone as a representative of the business.
 - ▶ It is unsullied with concerns about databases, user interfaces, or third-party frameworks.
 - ▶ It could serve the business in any system, irrespective of how that system was presented, or how the data was stored.
 - ▶ The Entity is pure business and *nothing else*.
- ▶ All that is required is that you bind the Critical Business Data and the Critical Business Rules together in a single and separate software module.

Loan
- principle - rate - period
+ makePayment() + applyInterest() + chargeLateFee()

Use cases

- ▶ Not all business rules are as pure as Entities.
 - ▶ Some business rules define the way that an *automated* system operates.
 - ▶ These rules would not be used in a manual environment, because they make sense only as part of an automated system.
 - ▶ For example, imagine an application that is used by bank officers to create a new loan.
- ▶ This is a *use case*.
 - ▶ A use case is a description of the way that an automated system is used.
 - ▶ It specifies
 - ▶ the input to be provided by the user,
 - ▶ the output to be returned to the user, and
 - ▶ the processing steps involved in producing that output.
 - ▶ A use case describes *application-specific* business rules as opposed to the Critical Business Rules within the Entities.

Gather Contact Info for New Loan

Input: Name, Address, Birthdate, D.L. #, SSN, etc.
Output: Same info for readback + credit score.

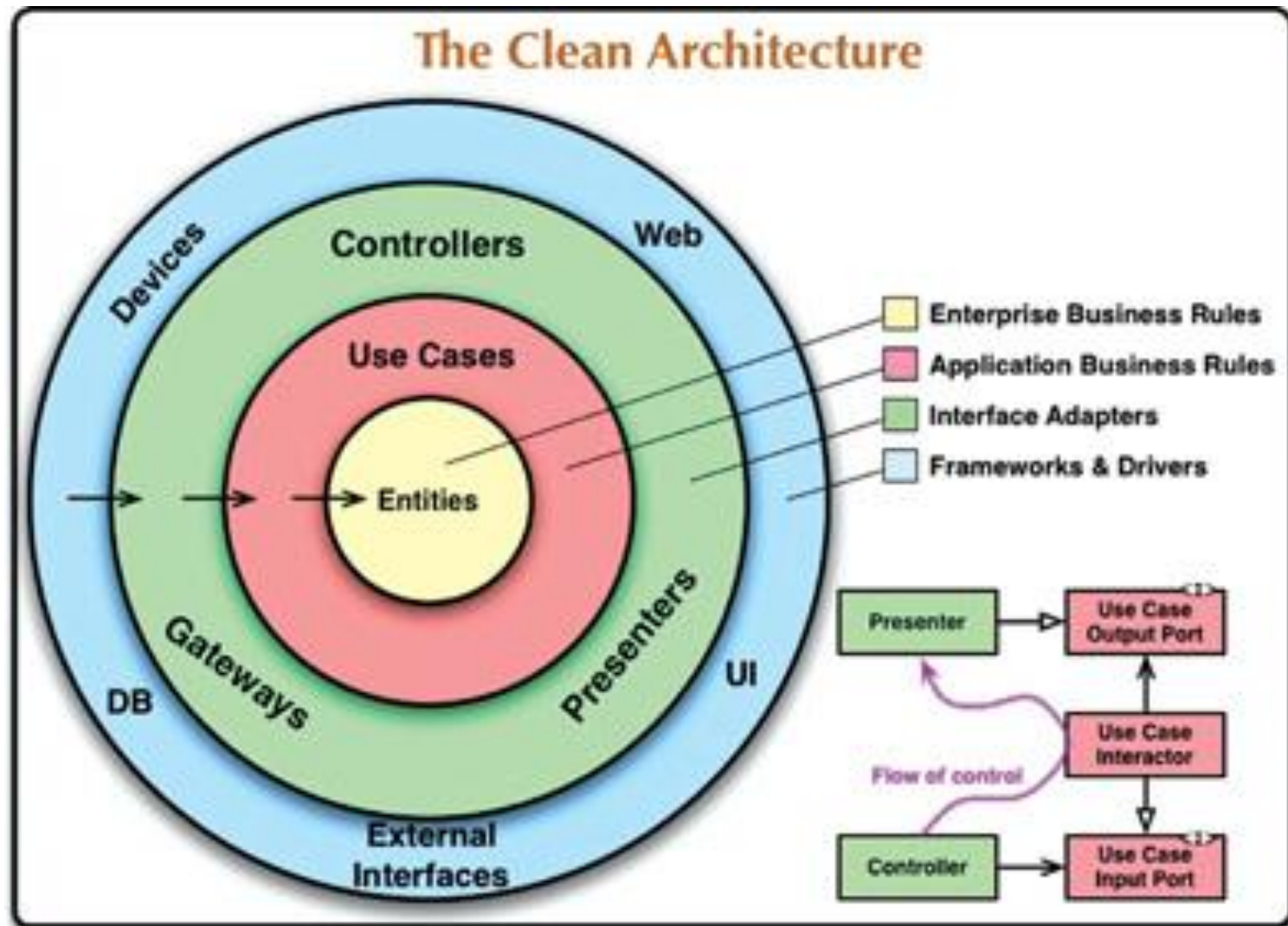
Primary Course:

1. Accept and validate name.
2. Validate address, birthdate, D.L.#, SSN, etc.
3. Get credit score.
4. If credit score is < 500 activate Denial.
5. Else create Customer and activate Loan Estimation.

Use cases

- ▶ Use cases contain the rules that specify how and when the Critical Business Rules within the Entities are invoked.
 - ▶ The use case does not describe the user interface (whether the application is delivered on the web, or on a console, or is a pure service).
 - ▶ Use cases do not describe how the system appears to the user.
 - ▶ How the data gets in and out of the system is irrelevant to the use cases.
- ▶ They are application-specific rules that govern the interaction between Users and Entities.
 - ▶ Entities have no knowledge of the use cases that control them.
 - ▶ High-level concepts, such as Entities, know nothing of lower-level concepts, such as use cases.
 - ▶ Lower-level use cases know about the higher-level Entities.
- ▶ Use cases are specific to an application and closer to the inputs and outputs of that system.
- ▶ Entities are generalizations that can be used in different applications, so they are farther from the inputs and outputs of the system.
 - ▶ Use cases depend on Entities;
 - ▶ Entities do not depend on use cases.

The Clean Architecture



Architecture features

- ▶ *Independent of frameworks.*
The architecture does not depend on the existence of some library of feature-laden software. This allows you to use such frameworks as tools, rather than forcing you to cram your system into their limited constraints.
- ▶ *Testable.* The business rules can be tested without the UI, database, web server, or any other external element.
- ▶ *Independent of the UI.* The UI can change easily, without changing the rest of the system. A web UI could be replaced with a console UI, for example, without changing the business rules.
- ▶ *Independent of the database.* Your business rules are not bound to the database.
- ▶ *Independent of any external agency.* In fact, your business rules don't know anything at all about the interfaces to the outside world.

The Clean Architecture

- ▶ The concentric circles represent different areas of software.
 - ▶ The further in you go, the higher level the software becomes.
 - ▶ The outer circles are mechanisms. The inner circles are policies.
- ▶ The overriding rule that makes this architecture work is the Dependency Rule:
 - ▶ Source code dependencies must point only inward, toward higher-level policies.
 - ▶ Nothing in an inner circle can know anything of an outer circle. Anything in an outer circle impacts the inner circles.
 - ▶ That includes functions, classes, variables, or any other named software entity.
- ▶ Entities encapsulate enterprise-wide Critical Business Rules and can be used by many different applications in the enterprise
- ▶ The software in the use cases layer contains application-specific business rules.
 - ▶ These use cases orchestrate the flow of data to and from the entities, and direct those entities to use their Critical Business Rules to achieve the goals of the use case.
 - ▶ We do not expect
 - ▶ changes in this layer to affect the entities.
 - ▶ this layer to be affected by changes to externalities such as the database, the UI, ...

The Clean Architecture

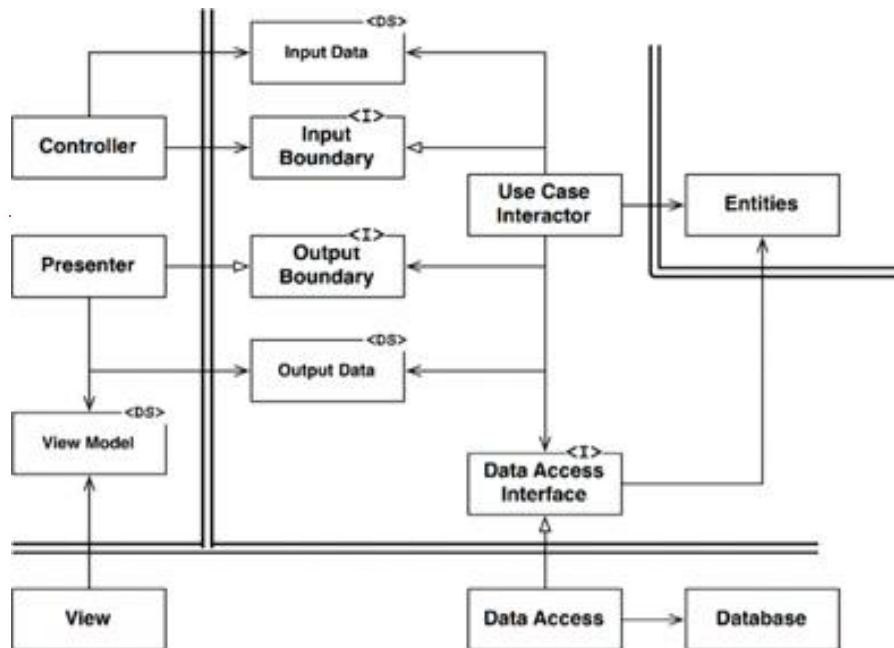
- ▶ The interface adapters layer converts data from the format used by the use cases and entities, to the format used by the external agency such as the database or the web.
 - ▶ It is this layer that will wholly contain the MVC architecture of a GUI.
 - ▶ The models are likely just data structures that are passed from the controllers to the use cases, and then back from the use cases to the presenters and views.
 - ▶ No code inward of this circle should know anything at all about the database
- ▶ The layer includes any other adapter necessary to convert data from some external form, such as an external service, to the internal form used by the use cases and entities.
- ▶ The outermost layer is composed of frameworks and tools such as the database and the web framework.
 - ▶ We don't write much code in this layer: only glue code that communicates to the next circle inward.
 - ▶ The frameworks and drivers layer is where all the details go.
 - ▶ The web is a detail. The database is a detail.

The Clean Architecture

- ▶ The Dependency Rule always applies.
 - ▶ Source code dependencies always point inward, where the level of abstraction and policy increases.
 - ▶ The outermost circle consists of low-level concrete details.
 - ▶ The innermost circle is the most general and highest level.
- ▶ At the lower right of the architecture diagram is shown how controllers and presenters are communicating with the use cases in the next layer.
- ▶ The flow of control begins in the controller, moves through the use case, and then winds up executing in the presenter.
 - ▶ The source code dependencies point inward toward the use cases.
 - ▶ We usually resolve this by using the Dependency Inversion Principle.
 - ▶ We would arrange interfaces and inheritance relationships such that the source code dependencies oppose the flow of control at just the right points across the boundary.
 - ▶ Suppose the use case needs to call the presenter. The call must not be direct because that would violate the Dependency Rule: No name in an outer circle can be mentioned by an inner circle.
 - ▶ So we call an interface in the inner circle, and have the presenter in the outer circle implement it.

The Clean Architecture

- ▶ The same technique is used to cross all the boundaries in the architectures.
 - ▶ We take advantage of dynamic polymorphism to create source code dependencies that oppose the flow of control so that we can conform to the Dependency Rule, no matter which direction the flow of control travels.
- ▶ Typically the data that crosses the boundaries consists of simple data structures.
 - ▶ The important thing is that isolated, simple data structures are passed across the boundaries.
 - ▶ We don't want to cheat and pass Entity objects or database rows.
 - ▶ We don't want the data structures to have any kind of dependency that violates the Dependency Rule.
- ▶ For example, many database frameworks return a convenient data format in response to a query.
 - ▶ We don't want to pass that row structure inward across a boundary. Doing so would violate the Dependency Rule because it would force an inner circle to know something about an outer circle.
 - ▶ Thus, when we pass data across a boundary, it is always in the form that is most convenient for the inner circle.



A typical scenario

A typical scenario for a web-based Java system using a database.

- ▶ The web server gathers input data from the user and hands it to the Controller on the upper left.
- ▶ The Controller packages that data into an object and passes it through the InputBoundary to the UseCaseInteractor.
- ▶ The UseCaseInteractor interprets that data and uses it to control the Entities.
- ▶ It uses the DataAccessInterface to bring the data used by those Entities into memory from the Database.
- ▶ Upon completion, the UseCaseInteractor gathers data from the Entities and constructs the OutputData.
- ▶ The OutputData is then passed through the OutputBoundary interface to the Presenter.
- ▶ The Presenter repackages the OutputData into viewable form as the ViewModel, another object.
- ▶ The View moves the data from the ViewModel into the HTML page.

The main component

- ▶ The Main component is the ultimate detail—the lowest-level policy. It is the initial entry point of the system.
 - ▶ Nothing, other than the operating system, depends on it.
 - ▶ Its job is to create all the Factories, Strategies, and other global facilities, and then hand control over to the high-level abstract portions of the system.
- ▶ It is in this Main component that dependencies should be injected by a Dependency Injection framework.
 - ▶ Once they are injected into Main, Main should distribute those dependencies normally, without using the framework.
- ▶ Think of Main as the dirtiest of all the dirty components.

Documenting the Software architecture

4 + 1 view model of software architecture

- ▶ A logical view, which shows the key abstractions in the system as objects or object classes.
- ▶ A process view, which shows how, at run-time, the system is composed of interacting processes.
- ▶ A development view, which shows how the software is decomposed for development.
- ▶ A physical view, which shows the system hardware and how software components are distributed across the processors in the system.
- ▶ Related using use cases or scenarios (+1)

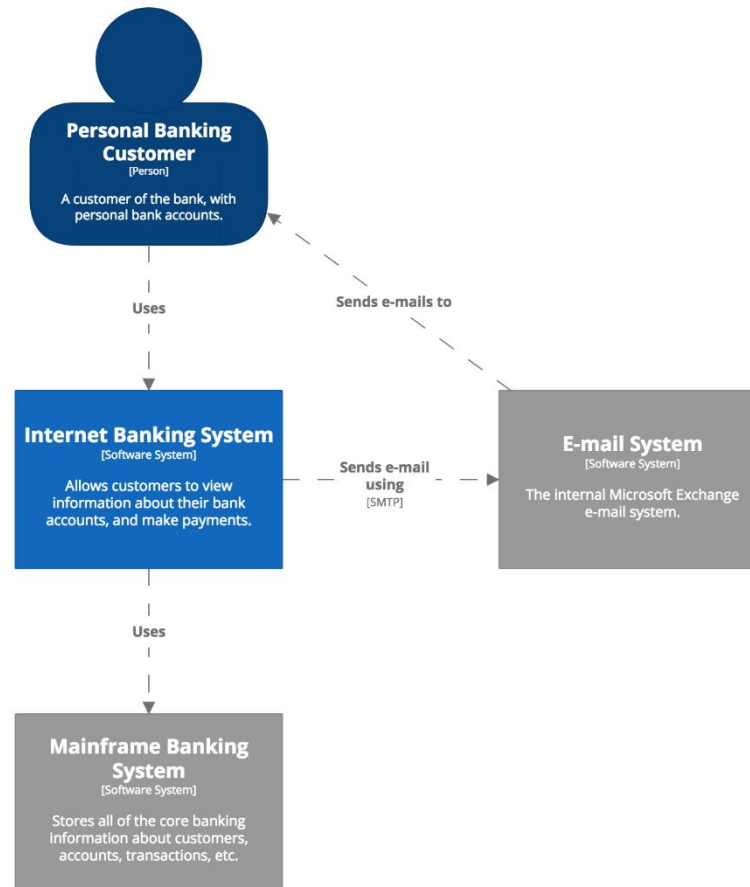
The C4 Model

1. Level 1: System Context diagram
2. Level 2: Container diagram
3. Level 3: Component diagram
4. Level 4: Code

Level 1: System Context diagram

- ▶ A System Context diagram is a good starting point for diagramming and documenting a software system, allowing you to step back and see the big picture.
 - ▶ Draw a diagram showing your system as a box in the centre, surrounded by its users and the other systems that it interacts with.
- ▶ Detail isn't important here as this is your zoomed out view showing a big picture of the system landscape. The focus should be on people (actors, roles, personas, etc) and software systems rather than technologies, protocols and other low-level details. It's the sort of diagram that you could show to non-technical people.
- ▶ **Scope:** A single software system.
- ▶ **Primary elements:** The software system in scope.
Supporting elements: People and software systems directly connected to the software system in scope.
- ▶ **Intended audience:** Everybody, both technical and non-technical people, inside and outside of the software development team.

Level 1: System Context diagram



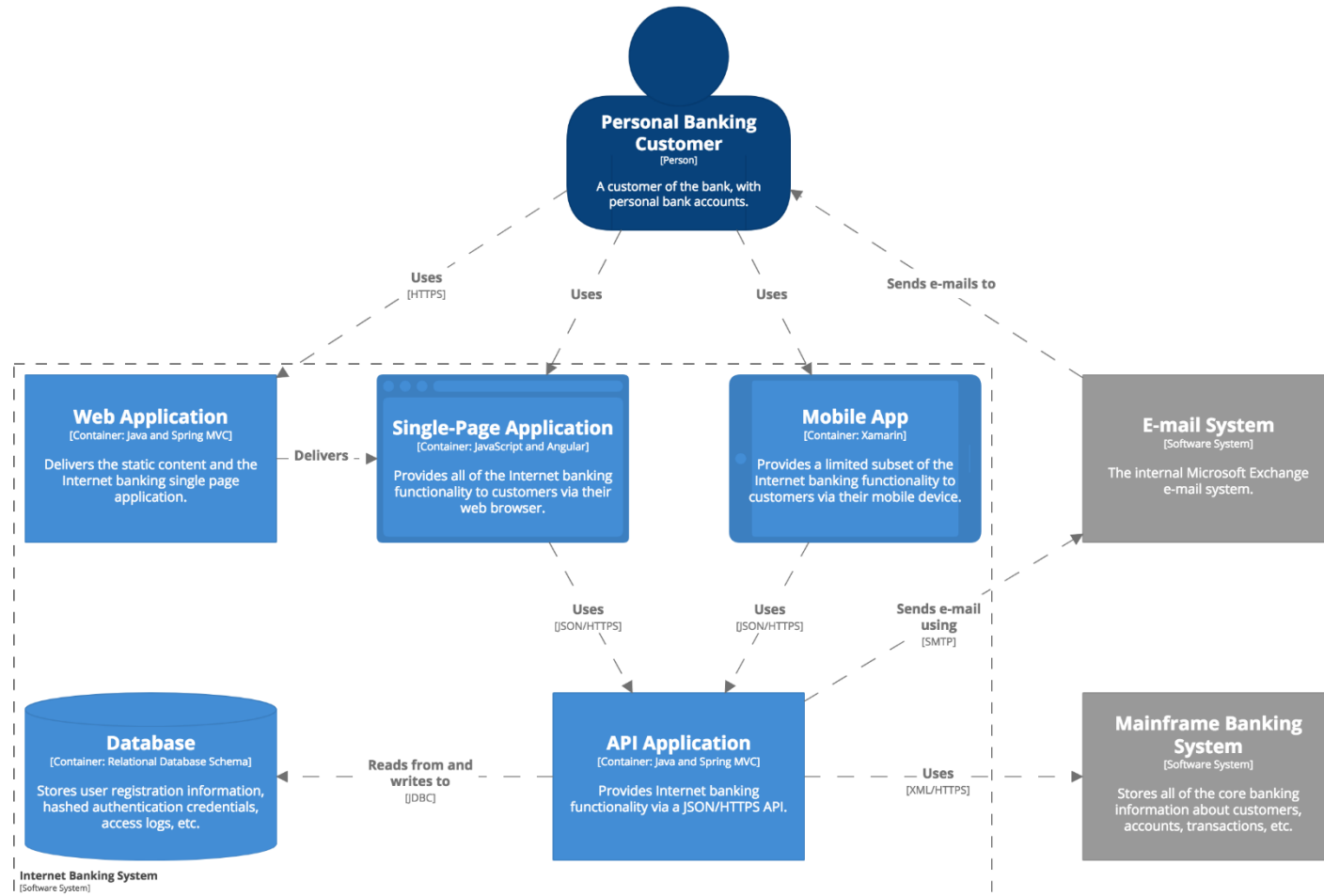
System Context diagram for Internet Banking System

The system context diagram for the Internet Banking System.
Last modified: Wednesday 02 May 2018 13:46 UTC

Level 2: Container diagram

- ▶ Once you understand how your system fits in to the overall IT environment, a really useful next step is to zoom-in to the system boundary with a Container diagram.
 - ▶ A "container" is something like a server-side web application, single-page application, desktop application, mobile app, database schema, file system, etc. Essentially, a container is a separately runnable/deployable unit (e.g. a separate process space) that executes code or stores data.
 - ▶ The Container diagram shows the high-level shape of the software architecture and how responsibilities are distributed across it. It also shows the major technology choices and how the containers communicate with one another.
 - ▶ It's a simple, high-level technology focussed diagram that is useful for software developers and support/operations staff alike.
- ▶ **Scope:** A single software system.
- ▶ **Primary elements:** Containers within the software system in scope.
Supporting elements: People and software systems directly connected to the containers.
- ▶ **Intended audience:** Technical people inside and outside of the software development team; including software architects, developers and operations/support staff.
- ▶ **Notes:** This diagram says nothing about deployment scenarios, clustering, replication, failover, etc.

Level 2: Container diagram



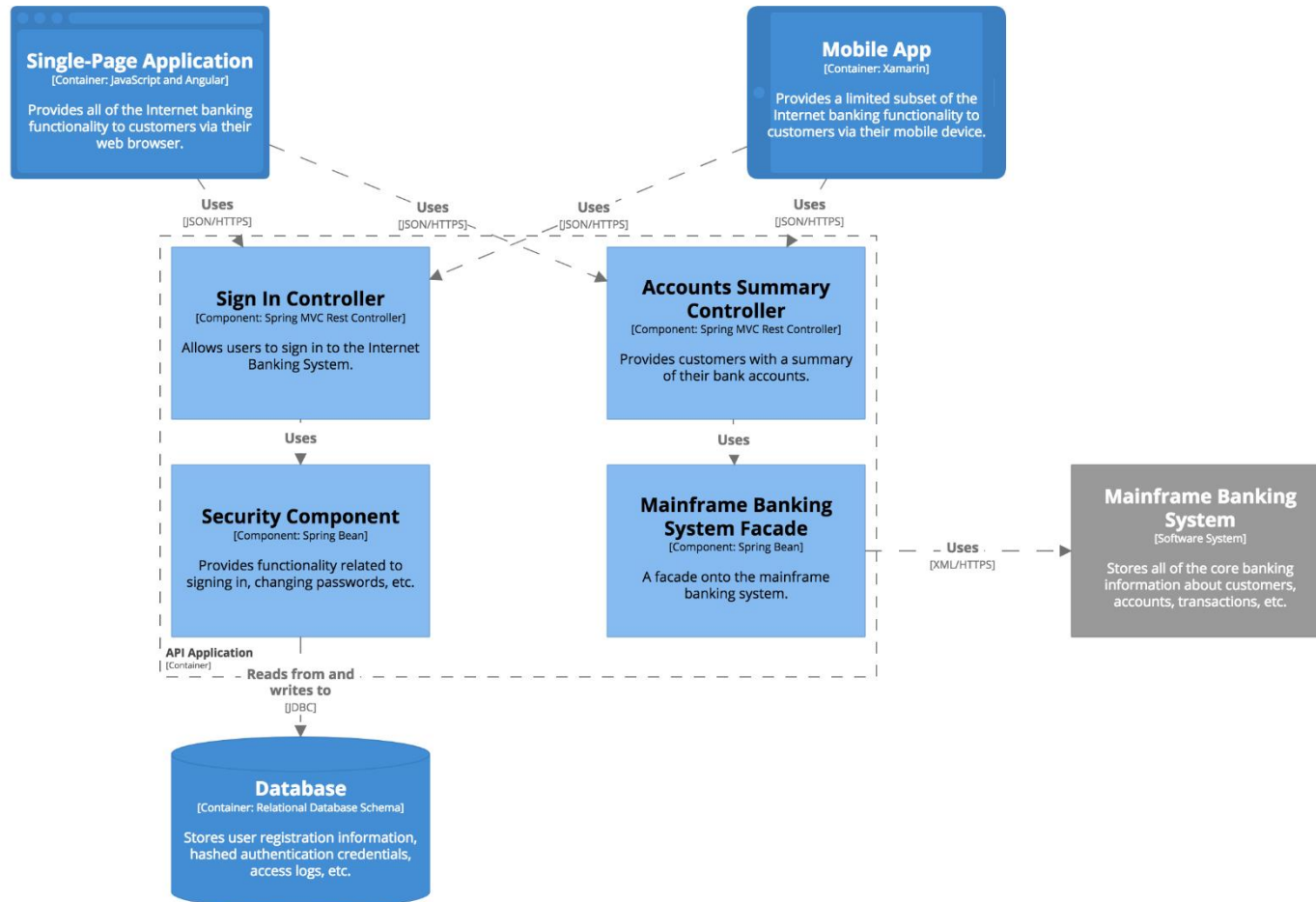
Container diagram for Internet Banking System

The container diagram for the Internet Banking System.
Last modified: Wednesday 02 May 2018 13:46 UTC

Level 3: Component diagram

- ▶ Next you can zoom in and decompose each container further to identify the major structural building blocks and their interactions.
- ▶ The Component diagram shows how a container is made up of a number of "components", what each of those components are, their responsibilities and the technology/implementation details.
- ▶ **Scope:** A single container.
- ▶ **Primary elements:** Components within the container in scope.
Supporting elements: Containers (within the software system in scope) plus people and software systems directly connected to the components.
- ▶ **Intended audience:** Software architects and developers.

Level 3: Component diagram



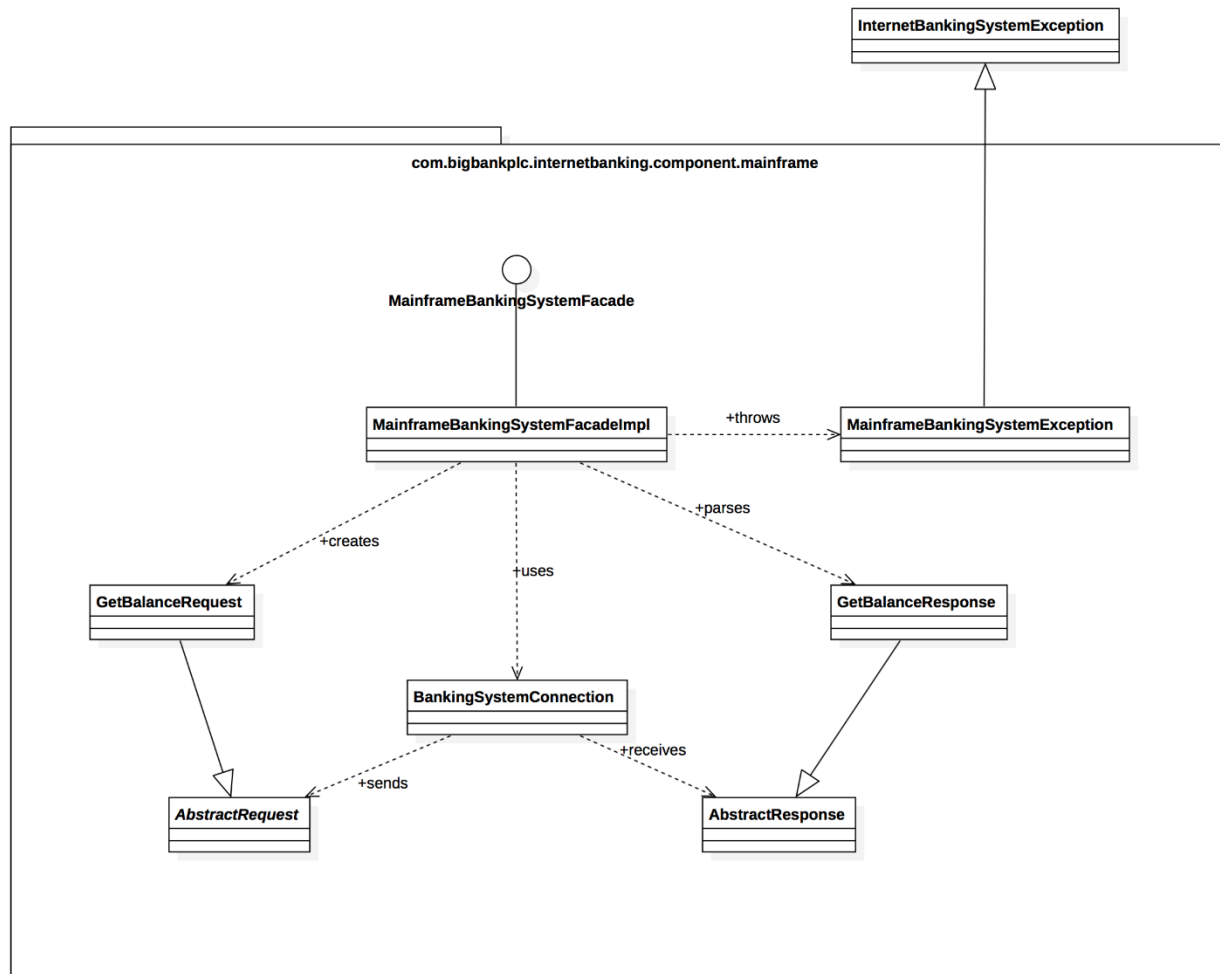
Component diagram for Internet Banking System - API Application

The component diagram for the API Application.
Last modified: Wednesday 02 May 2018 13:46 UTC

Level 4: Code

- ▶ Finally, you can zoom in to each component to show how it is implemented as code; using UML class diagrams, entity relationship diagrams or similar.
- ▶ This is an optional level of detail and is often available on-demand from tooling such as IDEs. Ideally this diagram would be automatically generated using tooling (e.g. an IDE or UML modelling tool), and you should consider showing only those attributes and methods that allow you to tell the story that you want to tell. This level of detail is not recommended for anything but the most important or complex components.
- ▶ **Scope:** A single component.
- ▶ **Primary elements:** Code elements (e.g. classes, interfaces, objects, functions, database tables, etc) within the component in scope.
- ▶ **Intended audience:** Software architects and developers.

Level 4: Code





References

- ▶ Frank Tsui, Orlando Karam, Barbara Bernal, Software engineering (Third Edition), Jones & Bartlett Learning
- ▶ Robert C. Martin: Clean Architecture: A Craftsman's Guide to Software Structure and Design, Prentice Hall; 2017
- ▶ Sommerville, Software engineering 9, Pearson
- ▶ Neal Ford, Mark Richards: Fundamentals of Software Architectures, O'Reilly 2021
- ▶ Raju Gandhi, Mark Richards, Neal Ford: Head First Software Architecture: A Learner's Guide to Architectural Thinking 1st Edition, 2024
- ▶ Software Architecture Fundamentals Understanding the Basics
- ▶ Neal Ford, Mark Richards: Fundamentals, Patterns, AntiPatterns, Soft Skills, Continuous Delivery, and Code Analysis Tools, O'Reilly Media March 2014
- ▶ Rachel Roumeliotis: Variations in event-driven architecture
<http://radar.oreilly.com/2015/02/variations-in-event-driven-architecture.html>
- ▶ <https://martinfowler.com/articles/serverless.html>

MATERIALE DIDATTICO INTERNO DEL CORSO DI Ingegneria del Software.

La diffusione e pubblicazione on line del materiale è assolutamente proibita.